

RotaBase — Arquitetura Completa de Sistema

Plataforma de Passeios de Aventura (Quadriciclos, UTVs, 4x4)

Stack: Next.js 14 (App Router) · Tailwind CSS · Supabase · Mercado Pago/Asaas · Evolution API/Z-API

Índice

1. [Schema SQL — Supabase](#)
 2. [Row Level Security \(RLS\) — Políticas de Acesso](#)
 3. [Funções e Triggers SQL](#)
 4. [Estrutura de Pastas — Next.js App Router](#)
 5. [Middleware de Isolamento/Marketplace](#)
 6. [Variáveis de Ambiente](#)
 7. [Tipos TypeScript Globais](#)
 8. [Componente: AgencyPage \(Página Pública da Agência\)](#)
 9. [Componente: BookingWizard \(Fluxo de Reserva\)](#)
 10. [Componente: WaiverForm \(Termo de Responsabilidade\)](#)
 11. [Componente: GuideDashboard \(Painel do Guia Offline\)](#)
 12. [Painel da Empresa — Settings Panel](#)
 13. [Server Actions — Reservas e Pagamentos](#)
 14. [Integração WhatsApp \(Evolution API / Z-API\)](#)
 15. [Integração de Pagamento \(Mercado Pago / Asaas\)](#)
 16. [Portal Pós-Reserva do Cliente](#)
 17. [Configuração PWA — Suporte Offline do Guia](#)
-

1. Schema SQL — Supabase

```

-- =====
-- ROTABASE – Schema SQL Completo para Supabase
-- =====

-- Habilitar extensões necessárias
CREATE EXTENSION IF NOT EXISTS "uuid-ossf";
CREATE EXTENSION IF NOT EXISTS "pg_trgm"; -- Busca por texto

-- =====
-- ENUMS
-- =====

CREATE TYPE user_role AS ENUM ('super_admin', 'empresa', 'guia', 'cliente');
CREATE TYPE vehicle_type AS ENUM ('quadriciclo', 'utv', 'jeep_4x4', 'buggy', 'outro');
CREATE TYPE booking_status AS ENUM ('pendente', 'confirmada', 'cancelada',
'concluida', 'no_show');
CREATE TYPE payment_policy AS ENUM ('sinal', 'integral');
CREATE TYPE payment_status AS ENUM ('aguardando', 'pago', 'estornado', 'falhou');
CREATE TYPE payment_method AS ENUM ('pix', 'cartao_credito', 'cartao_debito');
CREATE TYPE waiver_status AS ENUM ('pendente', 'assinado', 'recusado');
CREATE TYPE whatsapp_status AS ENUM ('enviado', 'falhou', 'pendente');
CREATE TYPE whatsapp_recipient_type AS ENUM ('cliente', 'proprietario', 'guia');

-- =====
-- TABELA: users (Estende auth.users do Supabase)
-- =====

CREATE TABLE public.users (
  id          UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,
  role        user_role NOT NULL DEFAULT 'cliente',
  full_name   TEXT NOT NULL,
  phone       TEXT,                                -- Usado para WhatsApp
  cpf         TEXT UNIQUE,                          -- Para o Termo de Responsabilidade
  rg          TEXT,
  avatar_url  TEXT,
  agency_id   UUID,                                -- FK para agencies (preenchido para
EMPRESA e GUIA)
  is_active   BOOLEAN NOT NULL DEFAULT TRUE,
  push_token  TEXT,                                -- Para notificações PWA/Push do Guia
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- =====
-- TABELA: agencies (Empresa parceira / Agência)
-- =====

CREATE TABLE public.agencies (
  id          UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  slug        TEXT NOT NULL UNIQUE, -- Ex: "alem-das-dunas"
  name        TEXT NOT NULL,
  description  TEXT,                -- História da empresa
  local_curiosities TEXT,          -- Curiosidades locais sobre a região
  logo_url    TEXT,
  banner_url  TEXT,

  -- Sede / Ponto de Partida
  hq_name     TEXT,                -- Ex: "Sede Além das Dunas"
  hq_address  TEXT,
  hq_lat      NUMERIC(10, 7),
  hq_lng      NUMERIC(10, 7),
  hq_google_maps_url TEXT,

```

```

    hq_waze_url          TEXT,
    hq_whatsapp_number   TEXT,          -- Número do proprietário para noti-
ficações

    -- Configurações de Pagamento
    payment_policy       payment_policy NOT NULL DEFAULT 'integral',
    deposit_type         TEXT CHECK (deposit_type IN ('percentual', 'valor_fixo')),
    deposit_value        NUMERIC(10, 2), -- % ou valor fixo conforme depos-
it_type

    -- Configurações de WhatsApp
    whatsapp_api_provider TEXT CHECK (whatsapp_api_provider IN ('evolution', 'zapi', 'no
ne')) DEFAULT 'none',
    whatsapp_api_url      TEXT,
    whatsapp_api_key      TEXT,          -- Armazenado encriptado (Supabase
Vault)
    whatsapp_instance     TEXT,          -- Nome da instância (Evolution API)
    notify_guide          BOOLEAN NOT NULL DEFAULT FALSE,

    -- Configurações de Pagamento Gateway
    payment_gateway       TEXT CHECK (payment_gateway IN ('mercado_pago', 'asaas', 'none
')) DEFAULT 'none',
    gateway_public_key    TEXT,
    gateway_secret_key    TEXT,          -- Armazenado encriptado (Supabase
Vault)
    gateway_pix_key       TEXT,

    -- Galeria de Fotos da Frota
    fleet_gallery_urls    TEXT[] DEFAULT '{}',

    avg_rating            NUMERIC(3, 2) DEFAULT 0.0, -- Calculado via trigger
    total_reviews         INTEGER DEFAULT 0,
    is_active             BOOLEAN NOT NULL DEFAULT TRUE,
    created_at            TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at            TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- FK de users para agencies
ALTER TABLE public.users
    ADD CONSTRAINT fk_users_agency
    FOREIGN KEY (agency_id) REFERENCES public.agencies(id) ON DELETE SET NULL;

-- =====
-- TABELA: agency_amenities (Comodidades da Sede)
-- =====

CREATE TABLE public.agency_amenities (
    id          UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    agency_id   UUID NOT NULL REFERENCES public.agencies(id) ON DELETE CASCADE,
    icon        TEXT NOT NULL,    -- Nome do ícone (Lucide/Heroicons): "parking", "ice-
cream", etc.
    label       TEXT NOT NULL,    -- Ex: "Estacionamento Privativo"
    description TEXT,
    is_active   BOOLEAN NOT NULL DEFAULT TRUE,
    sort_order  INTEGER NOT NULL DEFAULT 0,
    created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- =====
-- TABELA: vehicles (Frota de Veículos)
-- =====

CREATE TABLE public.vehicles (

```

```

    id                UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    agency_id         UUID NOT NULL REFERENCES public.agencies(id) ON DELETE CASCADE,
    type              vehicle_type NOT NULL,
    model             TEXT NOT NULL,          -- Ex: "Yamaha Grizzly 700"
    license_plate     TEXT,
    capacity          INTEGER NOT NULL,       -- Pessoas por veículo (ex: 2 para
quadríciclo, 4 para UTV)
    photo_url         TEXT,
    description       TEXT,
    is_active         BOOLEAN NOT NULL DEFAULT TRUE,
    created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    CONSTRAINT chk_capacity CHECK (capacity >= 1 AND capacity <= 10)
);

-- =====
-- TABELA: tours (Passeios / Produtos)
-- =====

CREATE TABLE public.tours (
    id                UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    agency_id         UUID NOT NULL REFERENCES public.agencies(id) ON DELETE CASCADE,
    title             TEXT NOT NULL,
    description       TEXT,
    duration_minutes  INTEGER NOT NULL,       -- Duração em minutos
    price_per_person  NUMERIC(10, 2) NOT NULL,
    max_people        INTEGER NOT NULL,       -- Máximo de pessoas por lote
    cover_image_url   TEXT,
    gallery_urls      TEXT[] DEFAULT '{}',
    included_items    TEXT[] DEFAULT '{}',   -- 0 que está incluído
    requirements      TEXT[] DEFAULT '{}',   -- Requisitos (ex: CNH, capacete)
    meeting_point     TEXT,
    is_active         BOOLEAN NOT NULL DEFAULT TRUE,
    sort_order        INTEGER NOT NULL DEFAULT 0,
    created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at        TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Tabela de associação Passeio <-> Veículos disponíveis para aquele passeio
CREATE TABLE public.tour_vehicles (
    tour_id           UUID NOT NULL REFERENCES public.tours(id) ON DELETE CASCADE,
    vehicle_id        UUID NOT NULL REFERENCES public.vehicles(id) ON DELETE CASCADE,
    PRIMARY KEY (tour_id, vehicle_id)
);

-- =====
-- TABELA: tour_schedules (Lotes / Horários dos Passeios)
-- =====

CREATE TABLE public.tour_schedules (
    id                UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    tour_id           UUID NOT NULL REFERENCES public.tours(id) ON DELETE CASCADE,
    agency_id         UUID NOT NULL REFERENCES public.agencies(id) ON DELETE
CASCADE,
    schedule_date     DATE NOT NULL,
    start_time        TIME NOT NULL,
    end_time          TIME,
    max_people        INTEGER NOT NULL,       -- Capacidade máxima deste lote
    confirmed_people  INTEGER NOT NULL DEFAULT 0, -- Calculado via trigger
    price_override    NUMERIC(10, 2),       -- Permite preço diferente do tour
para este lote
    guide_id          UUID REFERENCES public.users(id) ON DELETE SET NULL,
    notes             TEXT,

```

```

is_active          BOOLEAN NOT NULL DEFAULT TRUE,
created_at         TIMESTAMPTZ NOT NULL DEFAULT NOW(),
updated_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
CONSTRAINT chk_confirmed_lte_max CHECK (confirmed_people <= max_people)
);

-- View calculada de disponibilidade de veículos por lote
CREATE VIEW public.schedule_vehicle_availability AS
SELECT
    ts.id AS schedule_id,
    ts.tour_id,
    ts.agency_id,
    ts.schedule_date,
    ts.start_time,
    ts.max_people,
    ts.confirmed_people,
    ts.max_people - ts.confirmed_people AS available_spots,
    v.id AS vehicle_id,
    v.type AS vehicle_type,
    v.model AS vehicle_model,
    v.capacity AS vehicle_capacity,
    -- Veículos comprometidos = ceil(pessoas confirmadas / capacidade do veículo)
    CEIL(ts.confirmed_people::FLOAT / NULLIF(v.capacity, 0)) AS vehicles_used,
    -- Veículos disponíveis deste tipo para este lote
    (SELECT COUNT(*) FROM public.vehicles vv
     WHERE vv.agency_id = ts.agency_id
           AND vv.type = v.type
           AND vv.is_active = TRUE) -
    CEIL(ts.confirmed_people::FLOAT / NULLIF(v.capacity, 0)) AS vehicles_available
FROM public.tour_schedules ts
JOIN public.tours t ON t.id = ts.tour_id
JOIN public.tour_vehicles tv ON tv.tour_id = ts.tour_id
JOIN public.vehicles v ON v.id = tv.vehicle_id
WHERE ts.is_active = TRUE;

-- =====
-- TABELA: bookings (Reservas)
-- =====

CREATE TABLE public.bookings (
    id                UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    booking_ref       TEXT NOT NULL UNIQUE,      -- Código legível (ex:
RTB-2024-00001)
    agency_id         UUID NOT NULL REFERENCES public.agencies(id) ON DELETE RE-
STRICT,
    tour_id           UUID NOT NULL REFERENCES public.tours(id) ON DELETE RESTRICT,
    schedule_id       UUID NOT NULL REFERENCES public.tour_schedules(id) ON DELETE R
ESTRICT,
    client_id         UUID NOT NULL REFERENCES public.users(id) ON DELETE RESTRICT,

    -- Passageiros
    num_people        INTEGER NOT NULL CHECK (num_people >= 1),
    passenger_names    TEXT[] DEFAULT '{}',      -- Nomes dos passageiros adicionais

    -- Financeiro
    price_per_person   NUMERIC(10, 2) NOT NULL,
    total_amount       NUMERIC(10, 2) NOT NULL,
    amount_paid        NUMERIC(10, 2) NOT NULL DEFAULT 0,
    payment_policy     payment_policy NOT NULL,
    deposit_amount     NUMERIC(10, 2),          -- Valor do sinal cobrado (se policy
= sinal)

    -- Status

```

```

status booking_status NOT NULL DEFAULT 'pendente',
payment_status payment_status NOT NULL DEFAULT 'aguardando',
payment_method payment_method,
gateway_payment_id TEXT, -- ID da transação no gateway (MP/As-
aas)
gateway_pix_qrcode TEXT, -- QR Code Pix (base64 ou URL)
gateway_pix_copy_paste TEXT, -- Código Pix copia e cola

-- Check-in
checked_in_at TIMESTAMPTZ,
checked_in_by UUID REFERENCES public.users(id),

-- Pós-passeio
photo_gallery_url TEXT, -- URL do álbum/galeria de fotos do
passeio
video_url TEXT,

notes TEXT,
created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Índices para performance
CREATE INDEX idx_bookings_agency_id ON public.bookings(agency_id);
CREATE INDEX idx_bookings_schedule_id ON public.bookings(schedule_id);
CREATE INDEX idx_bookings_client_id ON public.bookings(client_id);
CREATE INDEX idx_bookings_status ON public.bookings(status);
CREATE INDEX idx_bookings_booking_ref ON public.bookings(booking_ref);

-- =====
-- TABELA: waivers (Termos de Responsabilidade)
-- =====

CREATE TABLE public.waivers (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  booking_id UUID NOT NULL REFERENCES public.bookings(id) ON DELETE CASCADE,
  client_id UUID NOT NULL REFERENCES public.users(id) ON DELETE RESTRICT,
  agency_id UUID NOT NULL REFERENCES public.agencies(id) ON DELETE RESTRICT,

  -- Dados do signatário
  signer_name TEXT NOT NULL,
  signer_cpf TEXT NOT NULL,
  signer_rg TEXT,
  signer_birthdate DATE,
  signer_phone TEXT,

  -- Assinatura
  signature_data TEXT NOT NULL, -- Assinatura em base64 (canvas drawing)
  signature_ip TEXT,
  signed_at TIMESTAMPTZ,

  -- Status e PDF gerado
  status waiver_status NOT NULL DEFAULT 'pendente',
  waiver_pdf_url TEXT, -- URL do PDF gerado (Supabase Storage)
  waiver_template TEXT, -- Snapshot do texto do termo assinado

  -- Para passageiros adicionais (um waiver por pessoa)
  is_main_client BOOLEAN NOT NULL DEFAULT TRUE,
  passenger_index INTEGER DEFAULT 0,

  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

```

CREATE INDEX idx_waivers_booking_id ON public.waivers(booking_id);
CREATE INDEX idx_waivers_client_id ON public.waivers(client_id);

-- =====
-- TABELA: reviews (Avaliações)
-- =====

CREATE TABLE public.reviews (
  id          UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  agency_id   UUID NOT NULL REFERENCES public.agencies(id) ON DELETE CASCADE,
  tour_id     UUID REFERENCES public.tours(id) ON DELETE SET NULL,
  booking_id  UUID UNIQUE REFERENCES public.bookings(id) ON DELETE SET NULL,
  client_id   UUID NOT NULL REFERENCES public.users(id) ON DELETE RESTRICT,

  rating      INTEGER NOT NULL CHECK (rating >= 1 AND rating <= 5),
  comment     TEXT,
  is_verified BOOLEAN NOT NULL DEFAULT TRUE, -- TRUE = cliente fez a reserva
  is_visible  BOOLEAN NOT NULL DEFAULT TRUE,
  agency_reply TEXT,
  replied_at  TIMESTAMPTZ,

  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_reviews_agency_id ON public.reviews(agency_id);

-- =====
-- TABELA: whatsapp_logs (Log de Disparos WhatsApp)
-- =====

CREATE TABLE public.whatsapp_logs (
  id          UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  agency_id   UUID NOT NULL REFERENCES public.agencies(id) ON DELETE CASCADE,
  booking_id  UUID REFERENCES public.bookings(id) ON DELETE SET NULL,
  recipient_type whatsapp_recipient_type NOT NULL,
  recipient_phone TEXT NOT NULL,
  template_name TEXT NOT NULL, -- Ex: "booking_confirmation", "new_booking_alert"
  message_body TEXT NOT NULL, -- Mensagem enviada
  status      whatsapp_status NOT NULL DEFAULT 'pendente',
  provider     TEXT, -- 'evolution' ou 'zapi'
  provider_msg_id TEXT, -- ID da mensagem retornado pelo provider
  error_message TEXT,
  sent_at     TIMESTAMPTZ,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_whatsapp_logs_booking_id ON public.whatsapp_logs(booking_id);
CREATE INDEX idx_whatsapp_logs_agency_id ON public.whatsapp_logs(agency_id);

-- =====
-- TABELA: payment_transactions (Histórico de Transações)
-- =====

CREATE TABLE public.payment_transactions (
  id          UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  booking_id  UUID NOT NULL REFERENCES public.bookings(id) ON DELETE RESTRICT,
  agency_id   UUID NOT NULL REFERENCES public.agencies(id) ON DELETE RESTRICT,
  gateway     TEXT NOT NULL, -- 'mercado_pago' ou 'asaas'
  gateway_tx_id TEXT NOT NULL,
  gateway_status TEXT NOT NULL,
  amount      NUMERIC(10, 2) NOT NULL,

```

```
payment_method    payment_method NOT NULL,  
pix_qrcode        TEXT,  
pix_copy_paste    TEXT,  
pix_expiration    TIMESTAMPTZ,  
metadata          JSONB DEFAULT '{}',          -- Resposta completa do gateway  
created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
updated_at        TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);
```

2. Row Level Security (RLS) — Políticas de Acesso

```

-- =====
-- Habilitar RLS em todas as tabelas
-- =====

ALTER TABLE public.users          ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.agencies       ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.agency_amenities ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.vehicles       ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.tours          ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.tour_vehicles  ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.tour_schedules ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.bookings       ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.waivers        ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.reviews        ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.whatsapp_logs  ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.payment_transactions ENABLE ROW LEVEL SECURITY;

-- =====
-- Função auxiliar: pega a role do usuário logado
-- =====

CREATE OR REPLACE FUNCTION public.get_my_role()
RETURNS user_role LANGUAGE sql STABLE SECURITY DEFINER AS $$
    SELECT role FROM public.users WHERE id = auth.uid()
$$;

CREATE OR REPLACE FUNCTION public.get_my_agency_id()
RETURNS UUID LANGUAGE sql STABLE SECURITY DEFINER AS $$
    SELECT agency_id FROM public.users WHERE id = auth.uid()
$$;

-- =====
-- POLÍTICAS: users
-- =====

-- Usuário vê apenas seu próprio perfil; super_admin vê todos
CREATE POLICY "users_select_own" ON public.users
    FOR SELECT USING (
        id = auth.uid()
        OR public.get_my_role() = 'super_admin'
        OR (public.get_my_role() IN ('empresa', 'guia') AND agency_id = public.get_my_agency_id())
    );

CREATE POLICY "users_update_own" ON public.users
    FOR UPDATE USING (id = auth.uid() OR public.get_my_role() = 'super_admin');

CREATE POLICY "users_insert_super_admin" ON public.users
    FOR INSERT WITH CHECK (public.get_my_role() = 'super_admin');

-- =====
-- POLÍTICAS: agencies
-- =====

-- Leitura pública de agências ativas (marketplace e white-label)
CREATE POLICY "agencies_select_public" ON public.agencies
    FOR SELECT USING (is_active = TRUE OR public.get_my_role() = 'super_admin');

-- Empresa gerencia apenas a própria agency
CREATE POLICY "agencies_update_own" ON public.agencies
    FOR UPDATE USING (
        id = public.get_my_agency_id() AND public.get_my_role() = 'empresa'
    );

```

```

    OR public.get_my_role() = 'super_admin'
);

CREATE POLICY "agencies_insert_super_admin" ON public.agencies
  FOR INSERT WITH CHECK (public.get_my_role() = 'super_admin');

-- =====
-- POLÍTICAS: agency_amenities, vehicles, tours, tour_schedules
-- =====

-- Leitura pública
CREATE POLICY "amenities_select_public" ON public.agency_amenities
  FOR SELECT USING (is_active = TRUE OR public.get_my_role() = 'super_admin');

CREATE POLICY "vehicles_select_public" ON public.vehicles
  FOR SELECT USING (is_active = TRUE OR public.get_my_role() IN ('super_admin', 'empresa', 'guia'));

CREATE POLICY "tours_select_public" ON public.tours
  FOR SELECT USING (is_active = TRUE OR public.get_my_role() = 'super_admin');

CREATE POLICY "schedules_select_public" ON public.tour_schedules
  FOR SELECT USING (is_active = TRUE OR public.get_my_role() IN ('super_admin', 'empresa', 'guia'));

-- Escrita apenas por empresa da mesma agência ou super_admin
CREATE POLICY "amenities_write_empresa" ON public.agency_amenities
  FOR ALL USING (
    agency_id = public.get_my_agency_id() AND public.get_my_role() = 'empresa'
    OR public.get_my_role() = 'super_admin'
  );

CREATE POLICY "vehicles_write_empresa" ON public.vehicles
  FOR ALL USING (
    agency_id = public.get_my_agency_id() AND public.get_my_role() = 'empresa'
    OR public.get_my_role() = 'super_admin'
  );

CREATE POLICY "tours_write_empresa" ON public.tours
  FOR ALL USING (
    agency_id = public.get_my_agency_id() AND public.get_my_role() = 'empresa'
    OR public.get_my_role() = 'super_admin'
  );

CREATE POLICY "schedules_write_empresa" ON public.tour_schedules
  FOR ALL USING (
    agency_id = public.get_my_agency_id() AND public.get_my_role() = 'empresa'
    OR public.get_my_role() = 'super_admin'
  );

-- =====
-- POLÍTICAS: bookings
-- =====

-- Cliente vê apenas suas reservas; empresa/guia vê da própria agência
CREATE POLICY "bookings_select" ON public.bookings
  FOR SELECT USING (
    client_id = auth.uid()
    OR agency_id = public.get_my_agency_id() AND public.get_my_role() IN ('empresa', 'guia')
    OR public.get_my_role() = 'super_admin'
  );

```

```

CREATE POLICY "bookings_insert_client" ON public.bookings
  FOR INSERT WITH CHECK (client_id = auth.uid());

CREATE POLICY "bookings_update_empresa" ON public.bookings
  FOR UPDATE USING (
    agency_id = public.get_my_agency_id() AND public.get_my_role() IN ('empresa', 'gui
a')
    OR public.get_my_role() = 'super_admin'
    OR client_id = auth.uid() -- cliente pode cancelar
  );

-- =====
-- POLÍTICAS: waivers
-- =====

CREATE POLICY "waivers_select" ON public.waivers
  FOR SELECT USING (
    client_id = auth.uid()
    OR agency_id = public.get_my_agency_id() AND public.get_my_role() IN ('empresa', '
guia')
    OR public.get_my_role() = 'super_admin'
  );

CREATE POLICY "waivers_insert" ON public.waivers
  FOR INSERT WITH CHECK (client_id = auth.uid());

CREATE POLICY "waivers_update" ON public.waivers
  FOR UPDATE USING (
    client_id = auth.uid()
    OR agency_id = public.get_my_agency_id() AND public.get_my_role() IN ('empresa', '
guia')
    OR public.get_my_role() = 'super_admin'
  );

-- =====
-- POLÍTICAS: reviews
-- =====

CREATE POLICY "reviews_select_public" ON public.reviews
  FOR SELECT USING (is_visible = TRUE OR public.get_my_role() IN ('super_admin', 'empr
esa'));

CREATE POLICY "reviews_insert_client" ON public.reviews
  FOR INSERT WITH CHECK (client_id = auth.uid());

CREATE POLICY "reviews_update" ON public.reviews
  FOR UPDATE USING (
    client_id = auth.uid()
    OR agency_id = public.get_my_agency_id() AND public.get_my_role() = 'empresa'
    OR public.get_my_role() = 'super_admin'
  );

```

3. Funções e Triggers SQL

```

-- =====
-- TRIGGER: Gerar booking_ref automático
-- =====

CREATE OR REPLACE FUNCTION public.generate_booking_ref()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
DECLARE
    seq_num INTEGER;
    year_str TEXT;
BEGIN
    year_str := TO_CHAR(NOW(), 'YYYY');
    SELECT COUNT(*) + 1 INTO seq_num FROM public.bookings
        WHERE EXTRACT(YEAR FROM created_at) = EXTRACT(YEAR FROM NOW());
    NEW.booking_ref := 'RTB-' || year_str || '-' || LPAD(seq_num::TEXT, 5, '0');
    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_generate_booking_ref
BEFORE INSERT ON public.bookings
FOR EACH ROW EXECUTE FUNCTION public.generate_booking_ref();

-- =====
-- TRIGGER: Atualizar confirmed_people no lote ao confirmar/cancelar reserva
-- =====

CREATE OR REPLACE FUNCTION public.update_schedule_confirmed_people()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    -- Recalcula pessoas confirmadas no lote
    UPDATE public.tour_schedules
    SET confirmed_people = (
        SELECT COALESCE(SUM(num_people), 0)
        FROM public.bookings
        WHERE schedule_id = COALESCE(NEW.schedule_id, OLD.schedule_id)
        AND status IN ('pendente', 'confirmada')
    ),
    updated_at = NOW()
    WHERE id = COALESCE(NEW.schedule_id, OLD.schedule_id);
    RETURN COALESCE(NEW, OLD);
END;
$$;

CREATE TRIGGER trg_update_schedule_people
AFTER INSERT OR UPDATE OR DELETE ON public.bookings
FOR EACH ROW EXECUTE FUNCTION public.update_schedule_confirmed_people();

-- =====
-- TRIGGER: Recalcular avg_rating da agência ao inserir/atualizar review
-- =====

CREATE OR REPLACE FUNCTION public.update_agency_avg_rating()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    UPDATE public.agencies
    SET
        avg_rating = (
            SELECT ROUND(AVG(rating)::NUMERIC, 2)
            FROM public.reviews
            WHERE agency_id = COALESCE(NEW.agency_id, OLD.agency_id)
            AND is_visible = TRUE
        ),

```

```

total_reviews = (
    SELECT COUNT(*)
    FROM public.reviews
    WHERE agency_id = COALESCE(NEW.agency_id, OLD.agency_id)
    AND is_visible = TRUE
),
updated_at = NOW()
WHERE id = COALESCE(NEW.agency_id, OLD.agency_id);
RETURN COALESCE(NEW, OLD);
END;
$$;

CREATE TRIGGER trg_update_agency_rating
AFTER INSERT OR UPDATE OR DELETE ON public.reviews
FOR EACH ROW EXECUTE FUNCTION public.update_agency_avg_rating();

-- =====
-- TRIGGER: updated_at automático
-- =====

CREATE OR REPLACE FUNCTION public.set_updated_at()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    NEW.updated_at = NOW();
    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_users_updated_at BEFORE UPDATE ON public.users
FOR EACH ROW EXECUTE FUNCTION public.set_updated_at();
CREATE TRIGGER trg_agencies_updated_at BEFORE UPDATE ON public.agencies
FOR EACH ROW EXECUTE FUNCTION public.set_updated_at();
CREATE TRIGGER trg_vehicles_updated_at BEFORE UPDATE ON public.vehicles
FOR EACH ROW EXECUTE FUNCTION public.set_updated_at();
CREATE TRIGGER trg_tours_updated_at BEFORE UPDATE ON public.tours
FOR EACH ROW EXECUTE FUNCTION public.set_updated_at();
CREATE TRIGGER trg_schedules_updated_at BEFORE UPDATE ON public.tour_schedules
FOR EACH ROW EXECUTE FUNCTION public.set_updated_at();
CREATE TRIGGER trg_bookings_updated_at BEFORE UPDATE ON public.bookings
FOR EACH ROW EXECUTE FUNCTION public.set_updated_at();
CREATE TRIGGER trg_waivers_updated_at BEFORE UPDATE ON public.waivers
FOR EACH ROW EXECUTE FUNCTION public.set_updated_at();

-- =====
-- FUNÇÃO: Verificar disponibilidade antes de reservar (chamada pelo app)
-- =====

CREATE OR REPLACE FUNCTION public.check_schedule_availability(
    p_schedule_id UUID,
    p_num_people INTEGER
) RETURNS TABLE (
    available BOOLEAN,
    spots_remaining INTEGER,
    reason TEXT
) LANGUAGE plpgsql SECURITY DEFINER AS $$
DECLARE
    v_max_people INTEGER;
    v_confirmed INTEGER;
    v_spots_remaining INTEGER;
BEGIN
    SELECT max_people, confirmed_people
    INTO v_max_people, v_confirmed
    FROM public.tour_schedules

```

```

WHERE id = p_schedule_id AND is_active = TRUE;

IF NOT FOUND THEN
    RETURN QUERY SELECT FALSE, 0, 'Lote não encontrado ou inativo';
    RETURN;
END IF;

v_spots_remaining := v_max_people - v_confirmed;

IF v_spots_remaining < p_num_people THEN
    RETURN QUERY SELECT FALSE, v_spots_remaining,
        'Apenas ' || v_spots_remaining || ' vagas disponíveis neste horário';
    RETURN;
END IF;

RETURN QUERY SELECT TRUE, v_spots_remaining, 'Disponível';
END;
$$;

-- =====
-- DADOS INICIAIS: Comodidades padrão
-- =====

-- Inserir um super_admin via função de trigger ao criar auth.users
CREATE OR REPLACE FUNCTION public.handle_new_user()
RETURNS TRIGGER LANGUAGE plpgsql SECURITY DEFINER SET search_path = public AS $$
BEGIN
    INSERT INTO public.users (id, full_name, role)
    VALUES (
        NEW.id,
        COALESCE(NEW.raw_user_meta_data->>'full_name', NEW.email),
        COALESCE((NEW.raw_user_meta_data->>'role')::user_role, 'cliente')
    );
    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_on_auth_user_created
AFTER INSERT ON auth.users
FOR EACH ROW EXECUTE FUNCTION public.handle_new_user();

```

4. Estrutura de Pastas — Next.js App Router

```

rotabase/
├── public/
│   ├── manifest.json          # PWA manifest
│   ├── sw.js                  # Service Worker (gerado via next-pwa)
│   └── icons/
│       ├── icon-192.png
│       └── icon-512.png
├── src/
│   ├── app/
│   │   ├── layout.tsx        # Layout raiz (providers, fonts)
│   │   ├── page.tsx          # Página inicial do Marketplace Global
│   │   └── globals.css
│   │
│   │   ├── (marketplace)/    # Grupo: Rotas do Marketplace Global
│   │   │   ├── layout.tsx    # Header com logo RotaBase + nav global
│   │   │   ├── page.tsx      # Home: busca, destaque, passeios
│   │   │   ├── explorar/
│   │   │   │   ├── page.tsx  # Listagem geral de passeios
│   │   │   │   └── sobre/
│   │   │   │       └── page.tsx
│   │   │
│   │   │   ├── agencia/
│   │   │   │   ├── [slug]/
│   │   │   │   │   ├── layout.tsx    # Modo Isolado / White-Label
│   │   │   │   │   ├── page.tsx      # Layout sem marca RotaBase
│   │   │   │   │   └── passeio/
│   │   │   │   │       ├── [tourId]/
│   │   │   │   │       │   └── page.tsx  # Detalhe do passeio
│   │   │   │   │       └── reserva/
│   │   │   │   │           ├── [scheduleId]/
│   │   │   │   │           │   ├── page.tsx  # Fluxo de reserva
│   │   │   │   │           │   ├── termo/
│   │   │   │   │           │       └── page.tsx # Assinatura do Termo
│   │   │   │   │           └── confirmacao/
│   │   │   │   │               └── page.tsx # Confirmação + QR Code PIX
│   │   │   │
│   │   │   │   ├── minha-reserva/
│   │   │   │   │   ├── [bookingRef]/
│   │   │   │   │   │   └── page.tsx      # Portal Pós-Reserva do Cliente
│   │   │   │
│   │   │   │   ├── auth/
│   │   │   │   │   ├── login/
│   │   │   │   │   │   └── page.tsx
│   │   │   │   │   ├── cadastro/
│   │   │   │   │   │   └── page.tsx
│   │   │   │   │   └── callback/
│   │   │   │   │       └── route.ts      # OAuth callback do Supabase
│   │   │   │
│   │   │   │   ├── dashboard/
│   │   │   │   │   ├── layout.tsx      # Painel Protegido (requer auth)
│   │   │   │   │   │   └── empresa/
│   │   │   │   │   │       ├── page.tsx  # Dashboard principal da empresa
│   │   │   │   │   │       ├── configuracoes/
│   │   │   │   │   │       │   ├── page.tsx  # Settings Panel completo
│   │   │   │   │   │       ├── passeios/
│   │   │   │   │   │       │   ├── page.tsx  # Listagem de passeios
│   │   │   │   │   │       │   ├── novo/
│   │   │   │   │   │       │       └── page.tsx
│   │   │   │   │   │       │   └── [tourId]/
│   │   │   │   │   │       │       └── page.tsx  # Editar passeio

```

└─ lotes/	
└─┬─ page.tsx	# Calendário de lotes/horários
└─ frota/	
└─┬─ page.tsx	# Gerenciar veículos
└─ reservas/	
└─┬─ page.tsx	# Lista de reservas recebidas
└─ guias/	
└─┬─ page.tsx	# Gerenciar guias
└─ avaliacoes/	
└─┬─ page.tsx	# Moderar e responder avaliações
└─ guia/	# Role: guia (PWA offline-first)
└─┬─ page.tsx	# Lista do dia (principal)
└─┬─ passeio/	
└─┬─┬─ [scheduleId]/	
└─┬─┬─┬─ page.tsx	# Check-in de passageiros
└─ cliente/	# Role: cliente
└─┬─ page.tsx	# Minhas reservas
└─┬─ perfil/	
└─┬─┬─ page.tsx	
└─ super-admin/	# Role: super_admin
└─┬─ page.tsx	
└─┬─ agencias/	
└─┬─┬─ page.tsx	
└─┬─ usuarios/	
└─┬─┬─ page.tsx	
└─ api/	
└─┬─ webhooks/	
└─┬─┬─ mercadopago/	
└─┬─┬─┬─ route.ts	# Webhook de pagamento MP
└─┬─┬─ asaas/	
└─┬─┬─┬─ route.ts	# Webhook de pagamento Asaas
└─┬─ whatsapp/	
└─┬─┬─ send/	
└─┬─┬─┬─ route.ts	# Endpoint para disparar WhatsApp
└─ components/	
└─┬─ agency/	
└─┬─┬─ AgencyHero.tsx	# Banner + Logo + Rating
└─┬─┬─ AgencyAbout.tsx	# História + Curiosidades
└─┬─┬─ AgencyFleetGallery.tsx	# Galeria de fotos da frota
└─┬─┬─ AgencyHQMap.tsx	# Mapa interativo da sede
└─┬─┬─ AgencyAmenities.tsx	# Badges de comodidades
└─┬─┬─ AgencyTours.tsx	# Vitrine de passeios
└─┬─┬─ AgencyReviews.tsx	# Seção de avaliações
└─ booking/	
└─┬─ BookingWizard.tsx	# Fluxo completo de reserva
└─┬─ SchedulePicker.tsx	# Seleção de data/horário
└─┬─ PeoplePicker.tsx	# Quantidade de pessoas
└─┬─ PaymentStep.tsx	# Pagamento PIX/Cartão
└─┬─ BookingConfirmation.tsx	# Tela de confirmação
└─ waiver/	
└─┬─ WaiverForm.tsx	# Formulário + assinatura canvas
└─┬─ SignaturePad.tsx	# Canvas de assinatura
└─ guide/	
└─┬─ GuideDashboard.tsx	# Painel principal do guia
└─┬─ PassengerList.tsx	# Lista de passageiros (offline)

```

├── CheckInButton.tsx          # Botão de check-in
├── dashboard/
│   ├── settings/
│   │   ├── AgencySettingsPanel.tsx # Painel completo de config
│   │   ├── PaymentSettings.tsx
│   │   ├── WhatsAppSettings.tsx
│   │   └── AmenitiesEditor.tsx
│   └── Sidebar.tsx
├── ui/                        # Componentes base (shadcn/ui)
│   ├── Button.tsx
│   ├── Card.tsx
│   ├── StarRating.tsx
│   └── ...
├── lib/
│   ├── supabase/
│   │   ├── client.ts          # Supabase Browser Client
│   │   ├── server.ts         # Supabase Server Client (SSR)
│   │   └── middleware.ts      # Supabase session refresh
│   ├── payments/
│   │   ├── mercadopago.ts
│   │   └── asaas.ts
│   ├── whatsapp/
│   │   ├── evolution.ts
│   │   └── zapi.ts
│   └── utils.ts
├── actions/                  # Server Actions
│   ├── booking.actions.ts
│   ├── waiver.actions.ts
│   ├── agency.actions.ts
│   └── review.actions.ts
├── hooks/
│   ├── useOfflineSync.ts      # IndexedDB sync para guia
│   └── useAgencyContext.ts    # Detecta modo marketplace vs. isolado
├── types/
│   └── database.ts            # Tipos TypeScript gerados pelo Supabase
├── middleware.ts             # Detecção marketplace/white-label + auth
├── next.config.ts
├── tailwind.config.ts
└── package.json

```

5. Middleware de Isolamento/Marketplace

```
// middleware.ts
import { NextRequest, NextResponse } from 'next/server';
import { createServerClient } from '@supabase/ssr';

export async function middleware(request: NextRequest) {
  const { pathname } = request.nextUrl;

  // — 1. Refresh de sessão Supabase —————
  let response = NextResponse.next({ request });
  const supabase = createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
        getAll: () => request.cookies.getAll(),
        setAll: (cookiesToSet) => {
          cookiesToSet.forEach(({ name, value, options }) => {
            request.cookies.set(name, value);
            response.cookies.set(name, value, options);
          });
        },
      },
    },
  );

  const { data: { user } } = await supabase.auth.getUser();

  // — 2. Proteção das rotas do dashboard —————
  if (pathname.startsWith('/dashboard')) {
    if (!user) {
      return NextResponse.redirect(new URL('/auth/login', request.url));
    }

    // Proteção por role
    const { data: profile } = await supabase
      .from('users')
      .select('role')
      .eq('id', user.id)
      .single();

    const role = profile?.role;

    if (pathname.startsWith('/dashboard/empresa') && role !== 'empresa' && role !== 'super_admin') {
      return NextResponse.redirect(new URL('/dashboard', request.url));
    }
    if (pathname.startsWith('/dashboard/guia') && role !== 'guia' && role !== 'empresa' && role !== 'super_admin') {
      return NextResponse.redirect(new URL('/dashboard', request.url));
    }
    if (pathname.startsWith('/dashboard/super-admin') && role !== 'super_admin') {
      return NextResponse.redirect(new URL('/dashboard', request.url));
    }
  }

  // — 3. Injetar contexto de modo (marketplace vs. white-label) —————
  // As rotas /agencia/[slug] já isolam naturalmente pelo segmento de URL.
  // Injetamos um header para que o layout saiba o modo atual.
  if (pathname.startsWith('/agencia/')) {
    response.headers.set('x-rotabase-mode', 'white-label');
  } else {
    response.headers.set('x-rotabase-mode', 'marketplace');
  }
}
```

```

    }

    return response;
  }

  export const config = {
    matcher: [
      '/((?!_next/static|_next/image|favicon.ico|icons|manifest.json|sw.js).*)',
    ],
  };
};

```

6. Variáveis de Ambiente

```

# .env.local

# — Supabase —————
NEXT_PUBLIC_SUPABASE_URL=https://xxxxxxxxxxx.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=eyJhbGciOi...
SUPABASE_SERVICE_ROLE_KEY=eyJhbGciOi... # Apenas servidor

# — Pagamentos —————
# Mercado Pago
MERCADO_PAGO_ACCESS_TOKEN=APP_USR-...
NEXT_PUBLIC_MERCADO_PAGO_PUBLIC_KEY=APP_USR-...
MERCADO_PAGO_WEBHOOK_SECRET=...

# Asaas (alternativa ao MP)
ASAAS_API_KEY=...
ASAAS_API_URL=https://api.asaas.com/v3 # ou sandbox: https://sandbox.asaas.com/api/v3
ASAAS_WEBHOOK_TOKEN=...

# — WhatsApp —————
# Evolution API
EVOLUTION_API_URL=https://api.sua-instancia.com
EVOLUTION_API_KEY=...

# Z-API (alternativa)
ZAPI_INSTANCE_ID=...
ZAPI_TOKEN=...

# — App —————
NEXT_PUBLIC_APP_URL=https://rotabase.com.br
NEXT_PUBLIC_APP_NAME=RotaBase

```

7. Tipos TypeScript Globais

```
// src/types/database.ts
// (Normalmente gerado via: npx supabase gen types typescript --linked)

export type UserRole = 'super_admin' | 'empresa' | 'guia' | 'cliente';
export type VehicleType = 'quadriciclo' | 'utv' | 'jeep_4x4' | 'buggy' | 'outro';
export type BookingStatus = 'pendente' | 'confirmada' | 'cancelada' | 'concluida' | 'no_show';
export type PaymentPolicy = 'sinal' | 'integral';
export type PaymentStatus = 'aguardando' | 'pago' | 'estornado' | 'falhou';
export type PaymentMethod = 'pix' | 'cartao_credito' | 'cartao_debito';
export type WaiverStatus = 'pendente' | 'assinado' | 'recusado';
export type WhatsAppStatus = 'enviado' | 'falhou' | 'pendente';

export interface Agency {
  id: string;
  slug: string;
  name: string;
  description: string | null;
  local_curiosities: string | null;
  logo_url: string | null;
  banner_url: string | null;
  hq_name: string | null;
  hq_address: string | null;
  hq_lat: number | null;
  hq_lng: number | null;
  hq_google_maps_url: string | null;
  hq_waze_url: string | null;
  hq_whatsapp_number: string | null;
  payment_policy: PaymentPolicy;
  deposit_type: 'percentual' | 'valor_fixo' | null;
  deposit_value: number | null;
  notify_guide: boolean;
  fleet_gallery_urls: string[];
  avg_rating: number;
  total_reviews: number;
  is_active: boolean;
  created_at: string;
  updated_at: string;
  // Relações (opcionais, dependendo do join)
  amenities?: AgencyAmenity[];
  tours?: Tour[];
  reviews?: Review[];
}

export interface AgencyAmenity {
  id: string;
  agency_id: string;
  icon: string;
  label: string;
  description: string | null;
  is_active: boolean;
  sort_order: number;
}

export interface Vehicle {
  id: string;
  agency_id: string;
  type: VehicleType;
  model: string;
  license_plate: string | null;
  capacity: number;
  photo_url: string | null;
}
```

```

    description: string | null;
    is_active: boolean;
  }

  export interface Tour {
    id: string;
    agency_id: string;
    title: string;
    description: string | null;
    duration_minutes: number;
    price_per_person: number;
    max_people: number;
    cover_image_url: string | null;
    gallery_urls: string[];
    included_items: string[];
    requirements: string[];
    meeting_point: string | null;
    is_active: boolean;
    sort_order: number;
  }

  export interface TourSchedule {
    id: string;
    tour_id: string;
    agency_id: string;
    schedule_date: string;           // 'YYYY-MM-DD'
    start_time: string;             // 'HH:MM:SS'
    end_time: string | null;
    max_people: number;
    confirmed_people: number;
    price_override: number | null;
    guide_id: string | null;
    is_active: boolean;
    // Calculated
    available_spots?: number;
  }

  export interface Booking {
    id: string;
    booking_ref: string;
    agency_id: string;
    tour_id: string;
    schedule_id: string;
    client_id: string;
    num_people: number;
    passenger_names: string[];
    price_per_person: number;
    total_amount: number;
    amount_paid: number;
    payment_policy: PaymentPolicy;
    deposit_amount: number | null;
    status: BookingStatus;
    payment_status: PaymentStatus;
    payment_method: PaymentMethod | null;
    gateway_payment_id: string | null;
    gateway_pix_qrcode: string | null;
    gateway_pix_copy_paste: string | null;
    checked_in_at: string | null;
    checked_in_by: string | null;
    photo_gallery_url: string | null;
    video_url: string | null;
    notes: string | null;
    created_at: string;
  }

```

```

// Relações
tour?: Tour;
schedule?: TourSchedule;
client?: UserProfile;
waivers?: Waiver[];
}

export interface Waiver {
  id: string;
  booking_id: string;
  client_id: string;
  agency_id: string;
  signer_name: string;
  signer_cpf: string;
  signer_rg: string | null;
  signer_birthdate: string | null;
  signer_phone: string | null;
  signature_data: string; // base64
  signature_ip: string | null;
  signed_at: string | null;
  status: WaiverStatus;
  waiver_pdf_url: string | null;
  is_main_client: boolean;
  passenger_index: number;
}

export interface Review {
  id: string;
  agency_id: string;
  tour_id: string | null;
  booking_id: string | null;
  client_id: string;
  rating: number;
  comment: string | null;
  is_verified: boolean;
  is_visible: boolean;
  agency_reply: string | null;
  replied_at: string | null;
  created_at: string;
  // Relações
  client?: UserProfile;
}

export interface UserProfile {
  id: string;
  role: UserRole;
  full_name: string;
  phone: string | null;
  cpf: string | null;
  avatar_url: string | null;
  agency_id: string | null;
}

```

8. Componente: AgencyPage (Página Pública da Agência)

```
// src/app/agencia/[slug]/page.tsx
import { createServerClient } from '@lib/supabase/server';
import { notFound } from 'next/navigation';
import AgencyHero from '@components/agency/AgencyHero';
import AgencyAbout from '@components/agency/AgencyAbout';
import AgencyHQMap from '@components/agency/AgencyHQMap';
import AgencyAmenities from '@components/agency/AgencyAmenities';
import AgencyFleetGallery from '@components/agency/AgencyFleetGallery';
import AgencyTours from '@components/agency/AgencyTours';
import AgencyReviews from '@components/agency/AgencyReviews';
import type { Agency } from '@types/database';

interface Props {
  params: { slug: string };
}

export async function generateMetadata({ params }: Props) {
  const supabase = createServerClient();
  const { data } = await supabase
    .from('agencies')
    .select('name, description, banner_url')
    .eq('slug', params.slug)
    .eq('is_active', true)
    .single();
  if (!data) return {};
  return {
    title: `${data.name} | RotaBase`,
    description: data.description?.slice(0, 160),
    openGraph: { images: data.banner_url ? [data.banner_url] : [] },
  };
}

export default async function AgencyPage({ params }: Props) {
  const supabase = createServerClient();

  const { data: agency, error } = await supabase
    .from('agencies')
    .select(`
      *,
      amenities:agency_amenities(* ORDER BY sort_order ASC),
      tours(* ORDER BY sort_order ASC),
      reviews(
        *, client:users(full_name, avatar_url)
        ORDER BY created_at DESC LIMIT 10
      )
    `)
    .eq('slug', params.slug)
    .eq('is_active', true)
    .single();

  if (error || !agency) notFound();

  return (
    <main className="min-h-screen bg-stone-50">
      {/* Banner + Logo + Nota */}
      <AgencyHero agency={agency} />

      <div className="max-w-6xl mx-auto px-4 py-8 space-y-16">
        {/* História + Curiosidades */}
        <AgencyAbout agency={agency} />

        {/* Galeria da Frota */}

```

```

        {agency.fleet_gallery_urls?.length > 0 && (
            <AgencyFleetGallery images={agency.fleet_gallery_urls} agencyName={agency.name} />
        )}

        {/* Sede / Ponto de Partida */}
        <AgencyHQMap agency={agency} />

        {/* Comodidades */}
        {agency.amenities?.length > 0 && (
            <AgencyAmenities amenities={agency.amenities} />
        )}

        {/* Vitrine de Passeios */}
        <AgencyTours tours={agency.tours || []} agencySlug={params.slug} />

        {/* Avaliações */}
        <AgencyReviews
            reviews={agency.reviews || []}
            avgRating={agency.avg_rating}
            totalReviews={agency.total_reviews}
        />
    </div>
</main>
);
}

```

```
// src/components/agency/AgencyHero.tsx
'use client';
import Image from 'next/image';
import { Star } from 'lucide-react';
import type { Agency } from '@types/database';

interface Props { agency: Agency }

export default function AgencyHero({ agency }: Props) {
  return (
    <section className="relative w-full h-72 md:h-96 overflow-hidden">
      {/* Banner */}
      {agency.banner_url ? (
        <Image src={agency.banner_url} alt={agency.name} fill className="object-cover" priority />
      ) : (
        <div className="w-full h-full bg-gradient-to-br from-amber-700 to-orange-500" />
      )}
      <div className="absolute inset-0 bg-black/40" />

      {/* Overlay com Logo + Nome + Rating */}
      <div className="absolute inset-0 flex flex-col items-center justify-center text-white px-4">
        {agency.logo_url && (
          <div className="relative w-24 h-24 mb-4 rounded-full overflow-hidden border-4 border-white shadow-lg">
            <Image src={agency.logo_url} alt={`Logo ${agency.name}`} fill className="object-cover" />
          </div>
        )}
        <h1 className="text-3xl md:text-5xl font-extrabold text-center drop-shadow">{agency.name}</h1>
        {agency.total_reviews > 0 && (
          <div className="flex items-center gap-2 mt-3 bg-black/30 px-4 py-1.5 rounded-full">
            <Star className="w-5 h-5 text-yellow-400 fill-yellow-400" />
            <span className="font-bold text-lg">{agency.avg_rating.toFixed(1)}</span>
            <span className="text-white/70 text-sm">{agency.total_reviews} avaliações
          </div>
        )}
      </div>
    </section>
  );
}
```

```
// src/components/agency/AgencyHQMap.tsx
'use client';
import { useEffect, useRef } from 'react';
import { MapPin, Navigation, Map } from 'lucide-react';
import type { Agency } from '@types/database';

interface Props { agency: Agency }

export default function AgencyHQMap({ agency }: Props) {
  const mapRef = useRef<HTMLDivElement>(null);

  useEffect(() => {
    if (!mapRef.current || !agency.hq_lat || !agency.hq_lng) return;

    // Leaflet (sem API Key - OpenStreetMap)
    import('leaflet').then((L) => {
      // Evitar dupla inicialização em strict mode
      if (mapRef.current!.innerHTML !== '') return;
      const map = L.map(mapRef.current!).setView([agency.hq_lat!, agency.hq_lng!], 15)
;
      L.tileLayer('https://blog.openstreetmap.org/wp-content/uploads/2025/07/Screen-Shot-2025-07-22-at-3.24.35-PM-1.png', {
        attribution: '© OpenStreetMap contributors',
      }).addTo(map);
      L.marker([agency.hq_lat!, agency.hq_lng!])
        .addTo(map)
        .bindPopup(`<b>${agency.hq_name ?? agency.name}</b><br>${agency.hq_address ?? ''}`)
        .openPopup();
    });

    return () => {
      import('leaflet').then((L) => {
        // Cleanup do mapa ao desmontar
        const mapInstance = (mapRef.current as any)?._leaflet_map;
        if (mapInstance) mapInstance.remove();
      });
    };
  }, [agency]);

  if (!agency.hq_lat || !agency.hq_lng) return null;

  return (
    <section id="sede" className="space-y-4">
      <div className="flex items-center gap-2">
        <MapPin className="w-6 h-6 text-amber-600" />
        <h2 className="text-2xl font-bold text-stone-800">Nossa Sede  Ponto de Partida
a</h2>
      </div>

      {agency.hq_address && (
        <p className="text-stone-600 flex items-center gap-1">
          <MapPin className="w-4 h-4 text-stone-400" />
          {agency.hq_address}
        </p>
      )}

      <div>
        {agency.hq_address}
      </div>

      <div>
        {agency.hq_address}
      </div>
    </section>
  );
}
```

```

/>

{/* Botões de Navegação */}
<div className="flex flex-wrap gap-3">
  {agency.hq_google_maps_url && (
    <a
      href={agency.hq_google_maps_url}
      target="_blank"
      rel="noopener noreferrer"
      className="flex items-center gap-2 bg-blue-600 hover:bg-blue-700 text-
white font-semibold px-5 py-2.5 rounded-xl shadow transition"
    >
      <Map className="w-5 h-5" />
      Abrir no Google Maps
    </a>
  )}
  {agency.hq_waze_url && (
    <a
      href={agency.hq_waze_url}
      target="_blank"
      rel="noopener noreferrer"
      className="flex items-center gap-2 bg-sky-500 hover:bg-sky-600 text-white
font-semibold px-5 py-2.5 rounded-xl shadow transition"
    >
      <Navigation className="w-5 h-5" />
      Abrir no Waze
    </a>
  )}
</div>
</section>
);
}

```

```
// src/components/agency/AgencyAmenities.tsx
'use client';
import {
  Car, IceCream2, Utensils, ShowerHead, Wifi, Sofa,
  Camera, Shield, Coffee, Shirt, Phone, CircleCheck,
} from 'lucide-react';
import type { AgencyAmenity } from '@types/database';

// Mapa de ícones disponíveis
const ICON_MAP: Record<string, React.ComponentType<{ className?: string }>> = {
  parking: Car,
  'ice-cream': IceCream2,
  crepe: Utensils,
  shower: ShowerHead,
  wifi: Wifi,
  lounge: Sofa,
  camera: Camera,
  shield: Shield,
  coffee: Coffee,
  shirt: Shirt,
  phone: Phone,
  default: CircleCheck,
};

interface Props { amenities: AgencyAmenity[] }

export default function AgencyAmenities({ amenities }: Props) {
  return (
    <section id="comodidades" className="space-y-4">
      <h2 className="text-2xl font-bold text-stone-800">Comodidades da Sede</h2>
      <div className="grid grid-cols-2 sm:grid-cols-3 md:grid-cols-4 gap-3">
        {amenities.map((amenity) => {
          const IconComponent = ICON_MAP[amenity.icon] ?? ICON_MAP.default;
          return (
            <div
              key={amenity.id}
              title={amenity.description ?? undefined}
              className="flex flex-col items-center justify-center gap-2 bg-white border border-stone-200 rounded-2xl p-4 shadow-sm hover:shadow-md hover:border-amber-400 hover:-translate-y-0.5 transition cursor-default"
            >
              <IconComponent className="w-7 h-7 text-amber-600" />
              <span className="text-sm font-medium text-stone-700 text-center leading-tight">
                {amenity.label}
              </span>
            </div>
          );
        })}
      </div>
    </section>
  );
}
```

```
// src/components/agency/AgencyTours.tsx
'use client';
import { useState } from 'react';
import Link from 'next/link';
import Image from 'next/image';
import { Clock, Users, ChevronRight, Filter } from 'lucide-react';
import type { Tour } from '@types/database';

interface Props {
  tours: Tour[];
  agencySlug: string;
}

export default function AgencyTours({ tours, agencySlug }: Props) {
  const [search, setSearch] = useState('');

  const filtered = tours.filter((t) =>
    t.title.toLowerCase().includes(search.toLowerCase())
  );

  return (
    <section id="passeios" className="space-y-6">
      <div className="flex flex-col sm:flex-row sm:items-center sm:justify-between gap-3">
        <h2 className="text-2xl font-bold text-stone-800">Nossos Passeios</h2>
        <div className="relative max-w-xs w-full">
          <Filter className="absolute left-3 top-1/2 -translate-y-1/2 w-4 h-4 text-stone-400" />
          <input
            type="text"
            placeholder="Filtrar passeios..."
            value={search}
            onChange={(e) => setSearch(e.target.value)}
            className="w-full pl-9 pr-4 py-2 rounded-xl border border-stone-200 text-sm focus:outline-none focus:border-amber-400"
          />
        </div>
      </div>

      {filtered.length === 0 ? (
        <p className="text-stone-500 text-center py-12">Nenhum passeio encontrado.</p>
      ) : (
        <div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 gap-6">
          {filtered.map((tour) => (
            <Link
              key={tour.id}
              href={`/${agencySlug}/passeio/${tour.id}`}
              className="group bg-white rounded-2xl overflow-hidden shadow hover:shadow-xl transition border border-stone-100"
            >
              <div className="relative h-48 bg-stone-200">
                {tour.cover_image_url ? (
                  <Image src={tour.cover_image_url} alt={tour.title} fill className="object-cover" />
                ) : (
                  <div className="w-full h-full bg-gradient-to-br from-amber-200 to-orange-300 flex items-center justify-center">
                    <span className="text-amber-700 text-4xl"></span>
                  </div>
                )}
              </div>
            </Link>
          ))}
        </div>
      )}
    </section>
  );
}
```

```

className="absolute top-3 right-3 bg-amber-500 text-white text-sm font-bold px-3 py-1
rounded-full shadow">
    R$ {tour.price_per_person.toFixed(2).replace('.', ',')} / pessoa
  </div>
</div>

  { /* Info */ }
  <div className="p-4">
    <h3 className="font-bold text-stone-800 text-lg group-hover:text-
amber-600 transition line-clamp-2">
      {tour.title}
    </h3>
    <div className="flex items-center gap-4 mt-2 text-stone-500 text-sm">
      <span className="flex items-center gap-1">
        <Clock className="w-4 h-4" />
        {Math.floor(tour.duration_minutes / 60)}h{tour.duration_minutes %
60 > 0 ? `${tour.duration_minutes % 60}min` : ''}
      </span>
      <span className="flex items-center gap-1">
        <Users className="w-4 h-4" />
        Até {tour.max_people} pessoas
      </span>
    </div>
    <div className="mt-4 flex items-center justify-between">
      <span className="text-amber-600 font-semibold text-sm">Ver horários
disponíveis</span>
      <ChevronRight className="w-4 h-4 text-amber-500 group-hov-
er:translate-x-1 transition" />
    </div>
  </div>
</Link>
  )}
</div>
  )}
</section>
);
}

```

```
// src/components/agency/AgencyReviews.tsx
'use client';
import { Star } from 'lucide-react';
import Image from 'next/image';
import type { Review } from '@types/database';

interface Props {
  reviews: Review[];
  avgRating: number;
  totalReviews: number;
}

function StarDisplay({ rating }: { rating: number }) {
  return (
    <div className="flex gap-0.5">
      {[1,2,3,4,5].map((n) => (
        <Star
          key={n}
          className={`w-4 h-4 ${n <= rating ? 'text-yellow-400 fill-yellow-400' : 'text-stone-300 fill-stone-300'}`}
        />
      ))}
    </div>
  );
}

export default function AgencyReviews({ reviews, avgRating, totalReviews }: Props) {
  return (
    <section id="avaliacoes" className="space-y-6">
      {/* Header */}
      <div className="flex flex-col sm:flex-row sm:items-center gap-4">
        <h2 className="text-2xl font-bold text-stone-800">Avaliações</h2>
        {totalReviews > 0 && (
          <div className="flex items-center gap-3 bg-amber-50 border border-amber-200 rounded-2xl px-5 py-2">
            <span className="text-4xl font-extrabold text-amber-600">{avgRating.toFixed(1)}</span>
            <div>
              <StarDisplay rating={Math.round(avgRating)} />
            </div>
            <p className="text-stone-500 text-sm mt-0.5">{totalReviews} avaliações verificadas</p>
          </div>
        )}
      </div>

      {/* Cards */}
      {reviews.length === 0 ? (
        <p className="text-stone-500">Ainda não há avaliações. Seja o primeiro!</p>
      ) : (
        <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
          {reviews.map((review) => (
            <div key={review.id} className="bg-white rounded-2xl p-5 shadow-sm border border-stone-100 space-y-2">
              {/* Cliente */}
              <div className="flex items-center gap-3">
                {review.client?.avatar_url ? (
                  <Image
                    src={review.client.avatar_url}
                    alt={review.client.full_name}
                    width={36} height={36}
                    className="rounded-full object-cover"

```

```

        />
    ) : (
        <div className="w-9 h-9 rounded-full bg-amber-100 flex items-center
justify-center text-amber-700 font-bold text-sm">
            {review.client?.full_name?.charAt(0).toUpperCase()}
        </div>
    )}
    <div>
        <p className="font-semibold text-stone-800 text-sm">{review.client?.
full_name}</p>
        {review.is_verified && (
            <span className="text-green-600 text-xs font-medium">✔ Compra veri
ficada</span>
        )}
    </div>
    <div className="ml-auto">
        <StarDisplay rating={review.rating} />
    </div>
</div>

    {/* Comentário */}
    {review.comment && (
        <p className="text-stone-700 text-sm leading-relaxed">{review.comment}
    </p>
    )}

    {/* Resposta da agência */}
    {review.agency_reply && (
        <div className="bg-amber-50 rounded-xl p-3 border-l-4 border-am-
ber-400">
            <p className="text-amber-800 text-xs font-semibold mb-1">Resposta
da empresa:</p>
            <p className="text-amber-900 text-sm">{review.agency_reply}</p>
        </div>
    )}
    </div>
    )})
</div>
    )}
</section>
);
}

```

9. Componente: BookingWizard (Fluxo de Reserva)

```

    ))}
  </div>

  <div className="p-6">
    {step === 'schedule' && (
      <SchedulePicker
        tourId={tour.id}
        agencyId={agency.id}
        selected={booking.scheduleId}
        onSelect={(schedule) => {
          setBooking((b) => ({ ...b, scheduleId: schedule.id, schedule }));
          setStep('people');
        }}
      />
    )}

    {step === 'people' && (
      <PeoplePicker
        maxPeople={booking.schedule?.available_spots ?? tour.max_people}
        value={booking.numPeople}
        onChange={(n) => setBooking((b) => ({
          ...b,
          numPeople: n,
          passengerNames: Array(Math.max(0, n - 1)).fill(''),
        })))}
        passengerNames={booking.passengerNames}
        onPassengerNameChange={(idx, name) =>
          setBooking((b) => {
            const names = [...b.passengerNames];
            names[idx] = name;
            return { ...b, passengerNames: names };
          })
        }
        onNext={() => setStep('payment')}
        onBack={() => setStep('schedule')}
      />
    )}

    {step === 'payment' && (
      <PaymentStep
        tour={tour}
        agency={agency}
        booking={booking}
        totalAmount={totalAmount}
        amountDue={amountDue}
        onBack={() => setStep('people')}
        onSuccess={(bookingRef) => {
          router.push(`/agencia/${agency.slug}/reserva/${booking.scheduleId}/confirmacao?ref=${bookingRef}`);
        }}
      />
    )}
  </div>
</div>
);
}

```

```
// src/components/booking/SchedulePicker.tsx
'use client';
import { useEffect, useState } from 'react';
import { createBrowserClient } from '@lib/supabase/client';
import { Calendar, Clock, Users } from 'lucide-react';
import type { TourSchedule } from '@types/database';

interface Props {
  tourId: string;
  agencyId: string;
  selected: string | null;
  onSelect: (schedule: TourSchedule & { available_spots: number }) => void;
}

export default function SchedulePicker({ tourId, agencyId, selected, onSelect }: Props) {
  const supabase = createBrowserClient();
  const [schedules, setSchedules] = useState<(TourSchedule & { available_spots: number })[]>([]);
  const [loading, setLoading] = useState(true);
  const [selectedDate, setSelectedDate] = useState<string>('');

  useEffect(() => {
    async function load() {
      setLoading(true);
      const today = new Date().toISOString().split('T')[0];
      const query = supabase
        .from('tour_schedules')
        .select('*')
        .eq('tour_id', tourId)
        .eq('agency_id', agencyId)
        .eq('is_active', true)
        .gte('schedule_date', today)
        .order('schedule_date', { ascending: true })
        .order('start_time', { ascending: true });

      const { data } = await query;
      if (data) {
        setSchedules(data.map((s) => ({
          ...s,
          available_spots: s.max_people - s.confirmed_people,
        })));
      }
      setLoading(false);
    }
    load();
  }, [tourId, agencyId]);

  const uniqueDates = [...new Set(schedules.map((s) => s.schedule_date))];
  const filteredDate = selectedDate
    ? schedules.filter((s) => s.schedule_date === selectedDate)
    : schedules;

  if (loading) return <div className="text-center py-8 text-stone-400 animate-pulse">Buscando horários...</div>;
  if (schedules.length === 0) return <div className="text-center py-8 text-stone-500">Nenhum horário disponível no momento.</div>;

  return (
    <div className="space-y-4">
      <h3 className="font-bold text-stone-800 text-lg">Selecione um horário</h3>
    </div>
  );
}
```

```

    { /* Filtro por data */ }
    <div className="flex gap-2 overflow-x-auto pb-1">
      <button
        onClick={() => setSelectedDate('')}
        className={`whitespace-nowrap px-4 py-2 rounded-xl text-sm font-medium
border transition ${
          selectedDate === '' ? 'bg-amber-500 text-white border-amber-500' : 'bor-
border-stone-200 text-stone-600 hover:border-amber-300'
        }}`
      >
        Todos
      </button>
      {uniqueDates.map((date) => {
        const d = new Date(date + 'T00:00:00');
        return (
          <button
            key={date}
            onClick={() => setSelectedDate(date)}
            className={`whitespace-nowrap px-4 py-2 rounded-xl text-sm font-medium
border transition ${
              selectedDate === date ? 'bg-amber-500 text-white border-amber-500' : '
border-stone-200 text-stone-600 hover:border-amber-300'
            }}`
          >
            <span className="flex items-center gap-1">
              <Calendar className="w-3.5 h-3.5" />
              {d.toLocaleDateString('pt-BR', { day: '2-digit', month: 'short' })}
            </span>
          </button>
        );
      })}
    </div>

    { /* Lista de lotes */ }
    <div className="space-y-3">
      {filtered.map((schedule) => {
        const isFull = schedule.available_spots <= 0;
        const isSelected = selected === schedule.id;
        const d = new Date(schedule.schedule_date + 'T00:00:00');
        return (
          <button
            key={schedule.id}
            disabled={isFull}
            onClick={() => !isFull && onSelect(schedule)}
            className={`w-full text-left rounded-2xl border-2 p-4 transition ${
              isSelected ? 'border-amber-500 bg-amber-50' :
              isFull ? 'border-stone-200 bg-stone-50 opacity-50 cursor-not-al-
lowed' :
              'border-stone-200 hover:border-amber-300 hover:bg-amber-50/50'
            }}`
          >
            <div className="flex items-center justify-between">
              <div className="space-y-0.5">
                <p className="font-semibold text-stone-800">
                  {d.toLocaleDateString('pt-BR', { weekday: 'long', day: '2-digit',
month: 'long' })}
                </p>
                <p className="flex items-center gap-1 text-stone-600 text-sm">
                  <Clock className="w-3.5 h-3.5" />
                  {schedule.start_time.slice(0, 5)}
                  {schedule.end_time && ` - ${schedule.end_time.slice(0, 5)}`}
                </p>
              </div>
            </div>
          </button>
        );
      })}
    </div>

```

```

    <div className="text-right">
      <p className={`font-bold text-sm flex items-center gap-1 ${isFull ?
'text-red-500' : 'text-green-600'}}`>
        <Users className="w-3.5 h-3.5" />
        {isFull ? 'Esgotado' : `${schedule.available_spots} vagas`}
      </p>
      {schedule.price_override && (
        <p className="text-amber-600 font-semibold text-sm">
          R$ {schedule.price_override.toFixed(2).replace('.', ',')}
        </p>
      )}
    </div>
  </div>
</button>
);
}}
</div>
</div>
);
}

```

10. Componente: WaiverForm (Termo de Responsabilidade)

```
// src/components/waiver/WaiverForm.tsx
'use client';
import { useRef, useState } from 'react';
import SignaturePad from './SignaturePad';
import { signWaiver } from '@actions/waiver.actions';
import { Loader2, CheckCircle } from 'lucide-react';

interface Props {
  bookingId: string;
  agencyId: string;
  clientId: string;
  bookingRef: string;
  onComplete: () => void;
}

export default function WaiverForm({ bookingId, agencyId, clientId, bookingRef, onComplete }: Props) {
  const [loading, setLoading] = useState(false);
  const [signed, setSigned] = useState(false);
  const signatureRef = useRef<{ getDataURL: () => string; isEmpty: () => boolean } | null>(null);

  const [form, setForm] = useState({
    signer_name: '',
    signer_cpf: '',
    signer_rg: '',
    signer_birthdate: '',
    signer_phone: '',
    accepted: false,
  });

  async function handleSubmit(e: React.FormEvent) {
    e.preventDefault();
    if (!form.accepted) return alert('Você deve concordar com os termos.');
```

if (signatureRef.current?.isEmpty()) return alert('Por favor, assine o termo.');

```
    const signatureData = signatureRef.current!.getDataURL();
    setLoading(true);
    try {
      await signWaiver({
        bookingId,
        agencyId,
        clientId,
        ...form,
        signatureData,
      });
      setSigned(true);
      setTimeout(onComplete, 1500);
    } catch (err) {
      alert('Erro ao registrar assinatura. Tente novamente.');
```

finally {

```
      setLoading(false);
    }
  }

  if (signed) {
    return (
      <div className="text-center py-12 space-y-4">
        <CheckCircle className="w-16 h-16 text-green-500 mx-auto" />
        <h3 className="text-xl font-bold text-stone-800">Termo assinado com sucesso!</h3>
        <p className="text-stone-600">Seu comprovante de reserva está sendo
```

```

gerado...</p>
</div>
);
}

return (
  <form onSubmit={handleSubmit} className="space-y-6 max-w-lg mx-auto">
    <div className="bg-amber-50 border border-amber-200 rounded-2xl p-4 text-sm
text-amber-900 leading-relaxed space-y-2">
      <h3 className="font-bold text-base">Termo de Responsabilidade {bookingRef}</h3>
      <p>Declaro estar ciente dos riscos inerentes à atividade de passeio de aventura com veículos off-road, incluso mas não limitado a quedas, colisões e acidentes na trais do percurso. Comprometo-me a seguir todas as orientações do guia responsável, u tilizar os equipamentos de segurança fornecidos e não operar o veículo sob efeito de á lcool ou substâncias.</p>
      <p>Isento a empresa operadora e seus colaboradores de responsabilidade civil e m caso de acidentes decorrentes de ato próprio ou descumprimento das normas de segur ança.</p>
    </div>

    <div className="grid grid-cols-1 sm:grid-cols-2 gap-4">
      <div className="sm:col-span-2">
        <label className="block text-sm font-medium text-stone-700 mb-1">Nome Com-
pleto *</label>
        <input
          required
          value={form.signer_name}
          onChange={(e) => setForm({ ...form, signer_name: e.target.value })}
          className="w-full border border-stone-200 rounded-xl px-4 py-2.5 text-sm
focus:outline-none focus:border-amber-400"
          placeholder="Seu nome completo"
        />
      </div>
      <div>
        <label className="block text-sm font-medium text-stone-700 mb-1">CPF *</la-
bel>
        <input
          required
          value={form.signer_cpf}
          onChange={(e) => setForm({ ...form, signer_cpf: e.target.value })}
          className="w-full border border-stone-200 rounded-xl px-4 py-2.5 text-sm
focus:outline-none focus:border-amber-400"
          placeholder="000.000.000-00"
          maxLength={14}
        />
      </div>
      <div>
        <label className="block text-sm font-medium text-stone-700 mb-1">RG</label>
        <input
          value={form.signer_rg}
          onChange={(e) => setForm({ ...form, signer_rg: e.target.value })}
          className="w-full border border-stone-200 rounded-xl px-4 py-2.5 text-sm
focus:outline-none focus:border-amber-400"
          placeholder="0000000"
        />
      </div>
      <div>
        <label className="block text-sm font-medium text-stone-700 mb-1">Data de Nas
cimento</label>
        <input
          type="date"

```

```

        value={form.signer_birthdate}
        onChange={(e) => setForm({ ...form, signer_birthdate: e.target.value })}
        className="w-full border border-stone-200 rounded-xl px-4 py-2.5 text-sm
focus:outline-none focus:border-amber-400"
      />
    </div>
    <div>
      <label className="block text-sm font-medium text-stone-700 mb-1">Telefone</
label>
      <input
        type="tel"
        value={form.signer_phone}
        onChange={(e) => setForm({ ...form, signer_phone: e.target.value })}
        className="w-full border border-stone-200 rounded-xl px-4 py-2.5 text-sm
focus:outline-none focus:border-amber-400"
        placeholder="(85) 99999-9999"
      />
    </div>
  </div>

  {/* Assinatura Digital */}
  <div>
    <label className="block text-sm font-medium text-stone-700 mb-2">Assinatura Di
gital *</label>
    <SignaturePad ref={signatureRef} />
    <p className="text-xs text-stone-400 mt-1">Assine com o dedo (celular) ou mous
e (computador)</p>
  </div>

  {/* Checkbox de aceite */}
  <label className="flex items-start gap-3 cursor-pointer">
    <input
      type="checkbox"
      checked={form.accepted}
      onChange={(e) => setForm({ ...form, accepted: e.target.checked })}
      className="mt-0.5 w-4 h-4 accent-amber-500"
      required
    />
    <span className="text-sm text-stone-700">
      Li e concordo com os termos de responsabilidade acima. Confirmando que todos
os dados informados são verídicos.
    </span>
  </label>

  <button
    type="submit"
    disabled={loading}
    className="w-full bg-amber-500 hover:bg-amber-600 text-white font-bold py-3
rounded-2xl shadow transition disabled:opacity-60 flex items-center justify-center
gap-2"
  >
    {loading ? <<Loader2 className="w-5 h-5 animate-spin" /> Registrando...</> :
'Assinar e Confirmar Reserva'}
  </button>
</form>
);
}

```

```
// src/components/waiver/SignaturePad.tsx
'use client';
import { forwardRef, useEffect, useImperativeHandle, useRef, useState } from 'react';
import { Trash2 } from 'lucide-react';

export interface SignaturePadHandle {
  getDataURL: () => string;
  isEmpty: () => boolean;
  clear: () => void;
}

const SignaturePad = forwardRef<SignaturePadHandle>((_, ref) => {
  const canvasRef = useRef<HTMLCanvasElement>(null);
  const isDrawing = useRef(false);
  const [empty, setEmpty] = useState(true);

  useImperativeHandle(ref, () => ({
    getDataURL: () => canvasRef.current?.toDataURL('image/png') ?? '',
    isEmpty: () => empty,
    clear: () => {
      const canvas = canvasRef.current;
      if (!canvas) return;
      canvas.getContext('2d')?.clearRect(0, 0, canvas.width, canvas.height);
      setEmpty(true);
    },
  }));

  useEffect(() => {
    const canvas = canvasRef.current;
    if (!canvas) return;
    const ctx = canvas.getContext('2d')!;
    ctx.strokeStyle = '#1c1917';
    ctx.lineWidth = 2.5;
    ctx.lineCap = 'round';
    ctx.lineJoin = 'round';

    function getPos(e: MouseEvent | TouchEvent): { x: number; y: number } {
      const rect = canvas!.getBoundingClientRect();
      const source = 'touches' in e ? e.touches[0] : e;
      return { x: source.clientX - rect.left, y: source.clientY - rect.top };
    }

    function start(e: MouseEvent | TouchEvent) {
      e.preventDefault();
      isDrawing.current = true;
      const { x, y } = getPos(e);
      ctx.beginPath();
      ctx.moveTo(x, y);
    }

    function draw(e: MouseEvent | TouchEvent) {
      if (!isDrawing.current) return;
      e.preventDefault();
      const { x, y } = getPos(e);
      ctx.lineTo(x, y);
      ctx.stroke();
      setEmpty(false);
    }

    function stop() { isDrawing.current = false; }

    canvas.addEventListener('mousedown', start);
    canvas.addEventListener('mousemove', draw);
    canvas.addEventListener('mouseup', stop);
  });
});
```

```

canvas.addEventListener('touchstart', start, { passive: false });
canvas.addEventListener('touchmove', draw, { passive: false });
canvas.addEventListener('touchend', stop);

return () => {
  canvas.removeEventListener('mousedown', start);
  canvas.removeEventListener('mousemove', draw);
  canvas.removeEventListener('mouseup', stop);
  canvas.removeEventListener('touchstart', start);
  canvas.removeEventListener('touchmove', draw);
  canvas.removeEventListener('touchend', stop);
};
}, []);

return (
  <div className="relative border-2 border-stone-200 rounded-2xl overflow-hidden bg-stone-50">
    <canvas
      ref={canvasRef}
      width={600}
      height={160}
      className="w-full touch-none cursor-crosshair"
      style={{ height: 160 }}
    />
    <button
      type="button"
      onClick={() => {
        const canvas = canvasRef.current;
        if (!canvas) return;
        canvas.getContext('2d')?.clearRect(0, 0, canvas.width, canvas.height);
        setEmpty(true);
      }}
      className="absolute top-2 right-2 text-stone-400 hover:text-red-500 transition"
      title="Limpar assinatura"
    >
      <Trash2 className="w-4 h-4" />
    </button>
    {empty && (
      <p
        className="absolute inset-0 flex items-center justify-center text-stone-300 pointer-events-none text-sm select-none">
        Assine aqui
      </p>
    )}
  </div>
);
});

SignaturePad.displayName = 'SignaturePad';
export default SignaturePad;

```

11. Componente: GuideDashboard (Painel do Guia Offline)

```
// src/components/guide/GuideDashboard.tsx
'use client';
import { useEffect, useState } from 'react';
import { createBrowserClient } from '@lib/supabase/client';
import PassengerList from './PassengerList';
import { useOfflineSync } from '@hooks/useOfflineSync';
import { Wifi, WifiOff, RefreshCw, CalendarDays } from 'lucide-react';
import type { Booking, TourSchedule } from '@types/database';

export default function GuideDashboard({ guideId }: { guideId: string }) {
  const supabase = createBrowserClient();
  const { isOnline, lastSync, syncToLocal } = useOfflineSync();
  const [schedules, setSchedules] = useState<TourSchedule & { bookings: Booking[]; tour_title: string }>[];
  const [selectedSchedule, setSelectedSchedule] = useState<string | null>(null);
  const [loading, setLoading] = useState(true);

  async function loadTodaySchedules() {
    setLoading(true);
    const today = new Date().toISOString().split('T')[0];
    const { data } = await supabase
      .from('tour_schedules')
      .select(`
        *,
        tour:tours(title),
        bookings(
          *,
          client:users(full_name, phone),
          waivers(status, signer_name)
        )
      `)
      .eq('guide_id', guideId)
      .eq('schedule_date', today)
      .order('start_time');

    if (data) {
      const mapped = data.map((s: any) => ({
        ...s,
        tour_title: s.tour?.title ?? 'Passeio',
        bookings: s.bookings ?? [],
      }));
      setSchedules(mapped);
      // Salva offline
      await syncToLocal('guide_schedules', mapped);
    }
    setLoading(false);
  }

  useEffect(() => { loadTodaySchedules(); }, []);

  const currentSchedule = schedules.find((s) => s.id === selectedSchedule);

  return (
    <div className="min-h-screen bg-stone-900 text-white px-4 py-6 space-y-6">
      { /* Header */ }
      <div className="flex items-center justify-between">
        <div>
          <h1 className="text-2xl font-bold">Painel do Guia</h1>
          <p className="text-stone-400 text-sm">
            {new Date().toLocaleDateString('pt-BR', { weekday: 'long', day: '2-digit', month: 'long' })}
          </p>
        </div>
      </div>
    </div>
  );
}
```

```

</div>
<div className="flex items-center gap-3">
  {isOnline ? (
    <span className="flex items-center gap-1 text-green-400 text-xs"><Wifi cla
ssName="w-4 h-4" /> Online</span>
  ) : (
    <span className="flex items-center gap-1 text-amber-400 text-xs"><WifiOff
className="w-4 h-4" /> Offline</span>
  )}
  <button onClick={loadTodaySchedules} disabled={!isOnline} className="text-
stone-400 hover:text-white transition disabled:opacity-40">
    <RefreshCw className="w-5 h-5" />
  </button>
</div>
</div>

{lastSync && (
  <p className="text-xs text-stone-500">
    Última sincronização: {new Date(lastSync).toLocaleTimeString('pt-BR')}
  </p>
)}

{/* Lista de passeios do dia */}
{!selectedSchedule ? (
  <div className="space-y-3">
    <h2 className="font-semibold text-stone-300 flex items-center gap-2">
      <CalendarDays className="w-4 h-4" /> Lotes de Hoje
    </h2>
    {loading ? (
      <div className="text-center py-8 text-stone-500 animate-pulse">Car-
regando...</div>
    ) : schedules.length === 0 ? (
      <div className="text-center py-12 text-stone-500">Nenhum lote agendado par
a hoje.</div>
    ) : (
      schedules.map((s) => (
        <button
          key={s.id}
          onClick={() => setSelectedSchedule(s.id)}
          className="w-full text-left bg-stone-800 hover:bg-stone-700 roun-
ded-2xl p-4 transition border border-stone-700"
        >
          <p className="font-bold text-white">{s.tour_title}</p>
          <p className="text-stone-400 text-sm">{s.start_time.slice(0, 5)} {s.
confirmed_people}/{s.max_people} pessoas</p>
          <div className="mt-2 flex gap-2">
            <span className="bg-stone-700 text-stone-300 text-xs px-2 py-0.5
rounded-full">
              {s.bookings.length} reservas
            </span>
            <span className={`text-xs px-2 py-0.5 rounded-full ${
              s.bookings.every((b: Booking) => b.checked_in_at)
                ? 'bg-green-900 text-green-400' : 'bg-amber-900 text-amber-400'
            }`} >
              {s.bookings.filter((b: Booking) => b.checked_in_at).length} check-
ins
            </span>
          </div>
        </button>
      ))
    )}
  </div>
) : (

```

```
currentSchedule && (  
  <PassengerList  
    schedule={currentSchedule}  
    bookings={currentSchedule.bookings}  
    onBack={() => setSelectedSchedule(null)}  
  />  
)  
  })  
</div>  
);  
}
```

```
// src/hooks/useOfflineSync.ts
'use client';
import { useEffect, useState } from 'react';

const DB_NAME = 'rotabase_guide';
const DB_VERSION = 1;

function openDB(): Promise<IDBDatabase> {
  return new Promise((resolve, reject) => {
    const req = indexedDB.open(DB_NAME, DB_VERSION);
    req.onupgradeneeded = (e) => {
      const db = (e.target as IDBOpenDBRequest).result;
      if (!db.objectStoreNames.contains('guide_schedules')) {
        db.createObjectStore('guide_schedules', { keyPath: 'id' });
      }
    };
    req.onsuccess = (e) => resolve((e.target as IDBOpenDBRequest).result);
    req.onerror = () => reject(req.error);
  });
}

export function useOfflineSync() {
  const [isOnline, setIsOnline] = useState(
    typeof window !== 'undefined' ? navigator.onLine : true
  );
  const [lastSync, setLastSync] = useState<string | null>(
    typeof window !== 'undefined' ? localStorage.getItem('guide_last_sync') : null
  );

  useEffect(() => {
    const handleOnline = () => setIsOnline(true);
    const handleOffline = () => setIsOnline(false);
    window.addEventListener('online', handleOnline);
    window.addEventListener('offline', handleOffline);
    return () => {
      window.removeEventListener('online', handleOnline);
      window.removeEventListener('offline', handleOffline);
    };
  }, []);

  async function syncToLocal(storeName: string, data: any[]) {
    const db = await openDB();
    const tx = db.transaction(storeName, 'readwrite');
    const store = tx.objectStore(storeName);
    store.clear();
    data.forEach((item) => store.put(item));
    const now = new Date().toISOString();
    localStorage.setItem('guide_last_sync', now);
    setLastSync(now);
  }

  async function getLocal<T>(storeName: string): Promise<T[]> {
    const db = await openDB();
    return new Promise((resolve, reject) => {
      const tx = db.transaction(storeName, 'readonly');
      const req = tx.objectStore(storeName).getAll();
      req.onsuccess = () => resolve(req.result as T[]);
      req.onerror = () => reject(req.error);
    });
  }
}
```

```
    return { isOnline, lastSync, syncToLocal, getLocal };  
}
```

12. Painel da Empresa — Settings Panel

```

<div className="flex border-b overflow-x-auto">
  {tabs.map((tab) => (
    <button
      key={tab.id}
      onClick={() => setActiveTab(tab.id)}
      className={`flex items-center gap-2 whitespace-nowrap px-5 py-3.5 text-sm
font-medium transition border-b-2 ${
        activeTab === tab.id
          ? 'border-amber-500 text-amber-600'
          : 'border-transparent text-stone-500 hover:text-stone-700'
        }`}
    >
      {tab.icon}
      {tab.label}
    </button>
  ))}
</div>

<div className="p-6">
  {/* — Tab: Perfil & Sede — */}
  {activeTab === 'perfil' && (
    <form onSubmit={handleSaveProfile} className="space-y-5 max-w-2xl">
      <div>
        <label className="block text-sm font-medium text-stone-700 mb-1">Nome
da Empresa</label>
        <input value={form.name} onChange={(e) => setForm({ ...form, name: e.tar
get.value }}}
          className="input-base" required />
        </div>

        <div>
          <label className="block text-sm font-medium text-stone-700 mb-1">Históri
a da Empresa</label>
          <textarea value={form.description} onChange={(e) => setForm({ ...form, d
escription: e.target.value }}}
            rows={4} className="input-base" placeholder="Conte a história da sua
empresa..." />
          </div>

          <div>
            <label className="block text-sm font-medium text-stone-700 mb-1">Curi-
osidades Locais</label>
            <textarea value={form.local_curiosities} onChange={(e) => setForm({ ...f
orm, local_curiosities: e.target.value }}}
              rows={3} className="input-base" placeholder="Curiosidades sobre a re-
gião, dunas, fauna..." />
            </div>

          <hr className="border-stone-100" />
          <h3 className="font-semibold text-stone-800">Sede / Ponto de Partida</h3>

          <div className="grid grid-cols-1 sm:grid-cols-2 gap-4">
            <div className="sm:col-span-2">
              <label className="block text-sm font-medium text-stone-700 mb-1">Nome
da Sede</label>
              <input value={form.hq_name} onChange={(e) => setForm({ ...form, hq_nam
e: e.target.value }}}
                className="input-base" placeholder="Ex: Sede Além das Dunas" />
              </div>
              <div className="sm:col-span-2">
                <label className="block text-sm font-medium text-stone-700 mb-1">En-
dereço Completo</label>
                <input value={form.hq_address} onChange={(e) => setForm({ ...form, hq_

```

```

address: e.target.value }}}
      className="input-base" placeholder="Rua, número, bairro, cidade" />
    </div>
  <div>
    <label className="block text-sm font-medium text-stone-700 mb-1">Latitude</label>
    <input type="number" step="any" value={form.hq_lat} onChange={(e) => setForm({ ...form, hq_lat: e.target.value })}
      className="input-base" placeholder="-3.732..." />
    </div>
    <div>
      <label className="block text-sm font-medium text-stone-700 mb-1">Longitude</label>
      <input type="number" step="any" value={form.hq_lng} onChange={(e) => setForm({ ...form, hq_lng: e.target.value })}
        className="input-base" placeholder="-38.512..." />
      </div>
      <div>
        <label className="block text-sm font-medium text-stone-700 mb-1">Link Google Maps</label>
        <input value={form.hq_google_maps_url} onChange={(e) => setForm({ ...form, hq_google_maps_url: e.target.value })}
          className="input-base" placeholder="https://maps.google.com/..." />
        </div>
        <div>
          <label className="block text-sm font-medium text-stone-700 mb-1">Link Waze</label>
          <input value={form.hq_waze_url} onChange={(e) => setForm({ ...form, hq_waze_url: e.target.value })}
            className="input-base" placeholder="https://waze.com/ul?..." />
          </div>
          <div>
            <label className="block text-sm font-medium text-stone-700 mb-1">WhatsApp do Proprietário</label>
            <input value={form.hq_whatsapp_number} onChange={(e) => setForm({ ...form, hq_whatsapp_number: e.target.value })}
              className="input-base" placeholder="5585999999999" />
            </div>
          </div>

      <button type="submit" disabled={saving}
        className="bg-amber-500 hover:bg-amber-600 text-white font-bold px-6 py-2.5 rounded-xl transition disabled:opacity-60 flex items-center gap-2">
        {saving ? <<Loader2 className="w-4 h-4 animate-spin" /> Salvando...</> : 'Salvar Alterações'}
      </button>
    </form>
  )}

  {activeTab === 'pagamento' && <PaymentSettings agency={agency} />}
  {activeTab === 'whatsapp' && <WhatsAppSettings agency={agency} />}
  {activeTab === 'comodidades' && <AmenitiesEditor agencyId={agency.id}
    initial={amenities} />}
  </div>
</div>
);
}

```

```
// src/components/dashboard/settings/PaymentSettings.tsx
'use client';
import { useState } from 'react';
import { updatePaymentSettings } from '@actions/agency.actions';
import { Loader2, Info } from 'lucide-react';
import type { Agency, PaymentPolicy } from '@types/database';

export default function PaymentSettings({ agency }: { agency: Agency }) {
  const [saving, setSaving] = useState(false);
  const [policy, setPolicy] = useState<PaymentPolicy>(agency.payment_policy);
  const [depositType, setDepositType] = useState<'percentual' | 'valor_fixo'>(
    agency.deposit_type ?? 'percentual'
  );
  const [depositValue, setDepositValue] =
    useState(agency.deposit_value?.toString() ?? '30');
  const [gateway, setGateway] = useState(agency.payment_gateway ?? 'none');

  async function handleSave(e: React.FormEvent) {
    e.preventDefault();
    setSaving(true);
    try {
      await updatePaymentSettings(agency.id, {
        payment_policy: policy,
        deposit_type: policy === 'sinal' ? depositType : null,
        deposit_value: policy === 'sinal' ? parseFloat(depositValue) : null,
        payment_gateway: gateway,
      });
      alert('Configurações de pagamento salvas!');
    } finally {
      setSaving(false);
    }
  }

  return (
    <form onSubmit={handleSave} className="space-y-6 max-w-lg">
      <div>
        <h3 className="font-semibold text-stone-800 mb-3">Política de Cobrança
Online</h3>
        <div className="space-y-3">
          {[
            { value: 'integral', label: 'Cobrar 100% no ato da reserva', desc: 'O cli-
ente paga o valor total online.' },
            { value: 'sinal', label: 'Cobrar apenas o Sinal (Entrada)', desc: 'O cli-
ente paga uma entrada. O restante paga no local.' },
          ] as const}.map((opt) => (
            <label key={opt.value}
              className={`flex gap-3 p-4 rounded-2xl border-2 cursor-pointer trans-
ition ${
                policy === opt.value ? 'border-amber-500 bg-amber-50' : 'border-
stone-200 hover:border-amber-300'
              }`}
            >
              <input type="radio" name="policy" value={opt.value}
                checked={policy === opt.value} onChange={() => setPolicy(opt.value)}
                className="mt-0.5 accent-amber-500" />
              <div>
                <p className="font-medium text-stone-800">{opt.label}</p>
                <p className="text-stone-500 text-sm">{opt.desc}</p>
              </div>
            </label>
          ))}
        </div>
      </div>
    </form>
  );
}
```

```

</div>

{/* Config do sinal */}
{policy === 'sinal' && (
  <div className="bg-amber-50 border border-amber-200 rounded-2xl p-4 space-
y-4">
    <p className="flex items-center gap-2 text-amber-800 text-sm font-medium">
      <Info className="w-4 h-4" /> Configure o valor do sinal
    </p>
    <div className="flex gap-3">
      <select value={depositType} onChange={(e) =>
setDepositType(e.target.value as any)}
        className="input-base flex-1">
        <option value="percentual">Percentual (%)</option>
        <option value="valor_fixo">Valor Fixo (R$)</option>
      </select>
      <div className="relative flex-1">
        <span
className="absolute left-3 top-1/2 -translate-y-1/2 text-stone-400 text-sm">
          {depositType === 'percentual' ? '%' : 'R$'}
        </span>
        <input type="number" min={1} step={depositType === 'percentual' ? 1 : 0.
01}
          value={depositValue} onChange={(e) => setDepositValue(e.target.value)}
          className="input-base pl-8" />
        </div>
      </div>
    </div>
  </div>
)}

{/* Gateway */}
<div>
  <h3 className="font-semibold text-stone-800 mb-3">Gateway de Pagamento</h3>
  <select value={gateway} onChange={(e) => setGateway(e.target.value)}
    className="input-base w-full">
    <option value="none">Selecionar...</option>
    <option value="mercado_pago">Mercado Pago</option>
    <option value="asaas">Asaas</option>
  </select>
  <p className="text-stone-500 text-sm mt-1">
    As chaves de API do gateway são configuradas nas variáveis de ambiente do se
rvidor.
  </p>
</div>

  <button type="submit" disabled={saving}
    className="bg-amber-500 hover:bg-amber-600 text-white font-bold px-6 py-2.5
rounded-xl transition disabled:opacity-60 flex items-center gap-2">
    {saving ? <<Loader2 className="w-4 h-4 animate-spin" /> Salvando...</> :
'Salvar Configurações'}
  </button>
</form>
);
}

```

```
// src/components/dashboard/settings/WhatsAppSettings.tsx
'use client';
import { useState } from 'react';
import { updateWhatsAppSettings } from '@actions/agency.actions';
import { Loader2, MessageCircle } from 'lucide-react';
import type { Agency } from '@types/database';

export default function WhatsAppSettings({ agency }: { agency: Agency }) {
  const [saving, setSaving] = useState(false);
  const [provider, setProvider] = useState(agency.whatsapp_api_provider ?? 'none');
  const [apiUrl, setApiUrl] = useState(agency.whatsapp_api_url ?? '');
  const [instance, setInstance] = useState(agency.whatsapp_instance ?? '');
  const [notifyGuide, setNotifyGuide] = useState(agency.notify_guide);

  async function handleSave(e: React.FormEvent) {
    e.preventDefault();
    setSaving(true);
    try {
      await updateWhatsAppSettings(agency.id, {
        whatsapp_api_provider: provider,
        whatsapp_api_url: apiUrl,
        whatsapp_instance: instance,
        notify_guide: notifyGuide,
      });
      alert('Configurações de WhatsApp salvas!');
    } finally {
      setSaving(false);
    }
  }

  return (
    <form onSubmit={handleSave} className="space-y-6 max-w-lg">
      <div className="flex items-center gap-2 text-stone-800 font-semibold">
        <MessageCircle className="w-5 h-5 text-green-500" />
        Notificações Automáticas via WhatsApp
      </div>

      <div>
        <label className="block text-sm font-medium text-stone-700 mb-2">Provedor de WhatsApp</label>
        <div className="grid grid-cols-3 gap-2">
          {[
            { value: 'none', label: 'Desativado' },
            { value: 'evolution', label: 'Evolution API' },
            { value: 'zapi', label: 'Z-API' },
          ] as const}.map((opt) => (
            <button key={opt.value} type="button" onClick={() => setProvider(opt.value)}
              className={`py-2.5 rounded-xl border-2 text-sm font-medium transition ${
                provider === opt.value ? 'border-green-500 bg-green-50 text-green-700' : 'border-stone-200 text-stone-600 hover:border-stone-300'
              }`}>
              {opt.label}
            </button>
          ))
        </div>
      </div>

      {provider !== 'none' && (
        <div>
          <label className="block text-sm font-medium text-stone-700 mb-1">URL da AP
```

```

I</label>
    <input value={apiUrl} onChange={(e) => setApiUrl(e.target.value)} class-
Name="input-base"
    placeholder={provider === 'evolution' ? 'https://api.seudominio.com' : '
https://api.z-api.io'} />
  </div>
  {provider === 'evolution' && (
    <div>
      <label className="block text-sm font-medium text-stone-700 mb-1">Nome
da Instância</label>
      <input value={instance} onChange={(e) => setInstance(e.target.value)} cl
assName="input-base"
      placeholder="minha-agencia" />
    </div>
  )}
  <div>
    <p className="text-stone-500 text-sm">
      ⚠ A chave de API (token) deve ser configurada via variável de ambiente
no servidor ({provider === 'evolution' ? 'EVOLUTION_API_KEY' : 'ZAPI_TOKEN'}).
    </p>
  </div>
</>
)}

{/* Notificar guia */}
<label className="flex items-start gap-3 cursor-pointer bg-stone-50 rounded-2xl
p-4 border border-stone-200">
  <input type="checkbox" checked={notifyGuide} onChange={(e) => setNotify-
Guide(e.target.checked)}
  className="mt-0.5 w-4 h-4 accent-green-500" />
  <div>
    <p className="font-medium text-stone-800">Notificar Guia em nova reserva</p>
    <p className="text-stone-500 text-sm">O guia designado ao lote receberá um W
hatsApp quando houver nova reserva.</p>
  </div>
</label>

  <button type="submit" disabled={saving}
  className="bg-green-600 hover:bg-green-700 text-white font-bold px-6 py-2.5
rounded-xl transition disabled:opacity-60 flex items-center gap-2">
    {saving ? <><Loader2 className="w-4 h-4 animate-spin" /> Salvando...</> :
'Salvar Configurações'}
  </button>
</form>
);
}

```

13. Server Actions — Reservas e Pagamentos

```
// src/actions/booking.actions.ts
'use server';
import { createServerClient } from '@lib/supabase/server';
import { createPaymentCharge } from '@lib/payments/mercadopago';
import { sendWhatsAppNotification } from '@lib/whatsapp/evolution';
import { revalidatePath } from 'next/cache';

interface CreateBookingInput {
  tourId: string;
  scheduleId: string;
  agencyId: string;
  numPeople: number;
  passengerNames: string[];
  paymentMethod: 'pix' | 'cartao_credito';
}

export async function createBooking(input: CreateBookingInput) {
  const supabase = createServerClient();

  // 1. Verificar sessão
  const { data: { user } } = await supabase.auth.getUser();
  if (!user) throw new Error('Usuário não autenticado');

  // 2. Verificar disponibilidade (função SQL)
  const { data: availability } = await supabase
    .rpc('check_schedule_availability', {
      p_schedule_id: input.scheduleId,
      p_num_people: input.numPeople,
    })
    .single();

  if (!availability?.available) {
    throw new Error(availability?.reason ?? 'Lote sem vagas disponíveis');
  }

  // 3. Buscar dados do tour e da agência
  const [{ data: tour }, { data: schedule }, { data: agency }] = await Promise.all([
    supabase.from('tours').select('*').eq('id', input.tourId).single(),
    supabase.from('tour_schedules').select('*').eq('id', input.scheduleId).single(),
    supabase.from('agencies').select('*').eq('id', input.agencyId).single(),
  ]);

  if (!tour || !schedule || !agency) throw new Error('Dados inválidos');

  const pricePerPerson = schedule.price_override ?? tour.price_per_person;
  const totalAmount = pricePerPerson * input.numPeople;

  let depositAmount: number;
  if (agency.payment_policy === 'integral') {
    depositAmount = totalAmount;
  } else {
    depositAmount = agency.deposit_type === 'percentual'
      ? totalAmount * ((agency.deposit_value ?? 30) / 100)
      : agency.deposit_value ?? 0;
  }

  // 4. Criar reserva (booking_ref gerado pelo trigger)
  const { data: booking, error: bookingError } = await supabase
    .from('bookings')
    .insert({
      agency_id: input.agencyId,
      tour_id: input.tourId,
    })

```

```

    schedule_id: input.scheduleId,
    client_id: user.id,
    num_people: input.numPeople,
    passenger_names: input.passengerNames,
    price_per_person: pricePerPerson,
    total_amount: totalAmount,
    payment_policy: agency.payment_policy,
    deposit_amount: depositAmount,
    payment_method: input.paymentMethod,
    status: 'pendente',
    payment_status: 'aguardando',
  })
  .select()
  .single();

if (bookingError || !booking) throw new Error('Erro ao criar reserva');

// 5. Gerar cobrança no gateway de pagamento
let pixQrcode: string | null = null;
let pixCopyPaste: string | null = null;
let gatewayPaymentId: string | null = null;

if (agency.payment_gateway === 'mercado_pago') {
  const charge = await createPaymentCharge({
    amount: depositAmount,
    method: input.paymentMethod,
    description: `${tour.title} - ${booking.booking_ref}`,
    externalRef: booking.id,
    payerEmail: user.email!,
  });
  gatewayPaymentId = charge.id;
  pixQrcode = charge.point_of_interaction?.transaction_data?.qr_code_base64 ?? null;
  pixCopyPaste = charge.point_of_interaction?.transaction_data?.qr_code ?? null;
}

// 6. Atualizar booking com dados do gateway
await supabase
  .from('bookings')
  .update({
    gateway_payment_id: gatewayPaymentId,
    gateway_pix_qrcode: pixQrcode,
    gateway_pix_copy_paste: pixCopyPaste,
  })
  .eq('id', booking.id);

// 7. Buscar dados do cliente para WhatsApp
const { data: clientProfile } = await supabase
  .from('users')
  .select('full_name, phone')
  .eq('id', user.id)
  .single();

// 8. Disparar WhatsApps (assíncrono, não bloqueia o retorno)
if (agency.whatsapp_api_provider !== 'none' && clientProfile?.phone) {
  void sendBookingWhatsApps({
    agency,
    booking: { ...booking, gateway_pix_copy_paste: pixCopyPaste },
    clientPhone: clientProfile.phone,
    clientName: clientProfile.full_name,
    tourTitle: tour.title,
    scheduleDate: schedule.schedule_date,
    scheduleTime: schedule.start_time,
  });
}

```

```

    }

    revalidatePath(`/agencia/${agency.slug}`);

    return {
      bookingRef: booking.booking_ref,
      bookingId: booking.id,
      pixQrcode,
      pixCopyPaste,
      amountDue: depositAmount,
      totalAmount,
      scheduleId: input.scheduleId,
    };
  }
}

async function sendBookingWhatsApps(params: {
  agency: any;
  booking: any;
  clientPhone: string;
  clientName: string;
  tourTitle: string;
  scheduleDate: string;
  scheduleTime: string;
}) {
  const { agency, booking, clientPhone, clientName, tourTitle, scheduleDate, scheduleTime } = params;

  const dateFormatted = new Date(scheduleDate + 'T00:00:00')
    .toLocaleDateString('pt-BR', { weekday: 'long', day: '2-digit', month: 'long' });

  const clientMsg = `🚗 *RotaBase – Confirmação de Reserva*\n\nOlá, ${clientName}!\nSua reserva foi confirmada:\n\n*Passeio:* ${tourTitle}\n*Data:* ${dateFormatted}\n*Horário:* ${scheduleTime.slice(0, 5)}\n*Código:* ${booking.booking_ref}\n\n📍 *Ponto de partida:* ${agency.hq_address}\n\n🗺️ Google Maps: ${agency.hq_google_maps_url ?? '-'}\n🚗 Waze: ${agency.hq_waze_url ?? '-'}\n\nNos vemos em breve! 🙌`;

  const ownerMsg = `🔔 *Nova Reserva Recebida!*\n\n*Passeio:* ${tourTitle}\n*Data:* ${dateFormatted} às ${scheduleTime.slice(0, 5)}\n*Cliente:* ${clientName}\n*Código:* ${booking.booking_ref}\n\nAcesse o painel para mais detalhes.`;

  await sendWhatsAppNotification({
    agency,
    phone: clientPhone,
    message: clientMsg,
    bookingId: booking.id,
    templateName: 'booking_confirmation',
    recipientType: 'cliente',
  });

  if (agency.hq_whatsapp_number) {
    await sendWhatsAppNotification({
      agency,
      phone: agency.hq_whatsapp_number,
      message: ownerMsg,
      bookingId: booking.id,
      templateName: 'new_booking_alert',
      recipientType: 'proprietario',
    });
  }
}

```

```
// src/actions/waiver.actions.ts
'use server';
import { createServerClient } from '@lib/supabase/server';
import { headers } from 'next/headers';
import { revalidatePath } from 'next/cache';

interface SignWaiverInput {
  bookingId: string;
  agencyId: string;
  clientId: string;
  signer_name: string;
  signer_cpf: string;
  signer_rg: string;
  signer_birthdate: string;
  signer_phone: string;
  signatureData: string;
}

export async function signWaiver(input: SignWaiverInput) {
  const supabase = createServerClient();

  const headersList = headers();
  const ip = headersList.get('x-forwarded-for') ?? 'unknown';

  const { data, error } = await supabase
    .from('waivers')
    .insert({
      booking_id: input.bookingId,
      client_id: input.clientId,
      agency_id: input.agencyId,
      signer_name: input.signer_name,
      signer_cpf: input.signer_cpf,
      signer_rg: input.signer_rg || null,
      signer_birthdate: input.signer_birthdate || null,
      signer_phone: input.signer_phone || null,
      signature_data: input.signatureData,
      signature_ip: ip,
      signed_at: new Date().toISOString(),
      status: 'assinado',
      is_main_client: true,
    })
    .select()
    .single();

  if (error) throw new Error('Erro ao registrar termo: ' + error.message);

  // Marcar booking como confirmado (após assinatura)
  await supabase
    .from('bookings')
    .update({ status: 'confirmada' })
    .eq('id', input.bookingId);

  revalidatePath('/minha-reserva');
  return data;
}
```

```
// src/actions/agency.actions.ts
'use server';
import { createServerClient } from '@lib/supabase/server';
import { revalidatePath } from 'next/cache';

export async function updateAgencySettings(agencyId: string, data: Record<string, any>) {
  const supabase = createServerClient();
  const { data: { user } } = await supabase.auth.getUser();
  if (!user) throw new Error('Não autorizado');

  const { error } = await supabase
    .from('agencies')
    .update(data)
    .eq('id', agencyId);

  if (error) throw new Error(error.message);
  revalidatePath('/dashboard/empresa/configuracoes');
}

export async function updatePaymentSettings(agencyId: string, data: {
  payment_policy: string;
  deposit_type: string | null;
  deposit_value: number | null;
  payment_gateway: string;
}) {
  return updateAgencySettings(agencyId, data);
}

export async function updateWhatsAppSettings(agencyId: string, data: {
  whatsapp_api_provider: string;
  whatsapp_api_url: string;
  whatsapp_instance: string;
  notify_guide: boolean;
}) {
  return updateAgencySettings(agencyId, data);
}
```

14. Integração WhatsApp (Evolution API / Z-API)

```
// src/lib/whatsapp/evolution.ts
import { createServerClient } from '@lib/supabase/server';

interface SendParams {
  agency: {
    id: string;
    whatsapp_api_provider: string;
    whatsapp_api_url: string;
    whatsapp_instance: string;
  };
  phone: string; // Formato: 5585999999999
  message: string;
  bookingId: string;
  templateName: string;
  recipientType: 'cliente' | 'proprietario' | 'guia';
}

export async function sendWhatsAppNotification(params: SendParams) {
  const { agency, phone, message, bookingId, templateName, recipientType } = params;
  const supabase = createServerClient();

  let providerMsgId: string | null = null;
  let status: 'enviado' | 'falhou' = 'falhou';
  let errorMsg: string | null = null;

  try {
    if (agency.whatsapp_api_provider === 'evolution') {
      const res = await fetch(
        `${agency.whatsapp_api_url}/message/sendText/${agency.whatsapp_instance}`,
        {
          method: 'POST',
          headers: {
            'Content-Type': 'application/json',
            apikey: process.env.EVOLUTION_API_KEY!,
          },
          body: JSON.stringify({
            number: phone,
            text: message,
          }),
        }
      );
      const json = await res.json();
      if (res.ok) {
        providerMsgId = json.key?.id ?? json.id ?? null;
        status = 'enviado';
      } else {
        errorMsg = JSON.stringify(json);
      }
    } else if (agency.whatsapp_api_provider === 'zapi') {
      const res = await fetch(
        `https://api.z-api.io/instances/${process.env.ZAPI_INSTANCE_ID}/token/${process.env.ZAPI_TOKEN}/send-text`,
        {
          method: 'POST',
          headers: { 'Content-Type': 'application/json' },
          body: JSON.stringify({ phone, message }),
        }
      );
      const json = await res.json();
      if (res.ok) {
        providerMsgId = json.messageId ?? null;
        status = 'enviado';
      }
    }
  } catch (error) {
    errorMsg = error.message;
  }
}
```

```
    } else {  
      errorMsg = JSON.stringify(json);  
    }  
  }  
} catch (err: any) {  
  errorMsg = err.message;  
}  
  
// Registrar log  
await supabase.from('whatsapp_logs').insert({  
  agency_id: agency.id,  
  booking_id: bookingId,  
  recipient_type: recipientType,  
  recipient_phone: phone,  
  template_name: templateName,  
  message_body: message,  
  status,  
  provider: agency.whatsapp_api_provider,  
  provider_msg_id: providerMsgId,  
  error_message: errorMsg,  
  sent_at: status === 'enviado' ? new Date().toISOString() : null,  
});  
  
return { status, providerMsgId };  
}
```

15. Integração de Pagamento (Mercado Pago / Asaas)

```
// src/lib/payments/mercadopago.ts
import MercadoPagoConfig, { Payment } from 'mercadopago';

const client = new MercadoPagoConfig({
  accessToken: process.env.MERCADO_PAGO_ACCESS_TOKEN!,
  options: { timeout: 5000 },
});

interface ChargeInput {
  amount: number;
  method: 'pix' | 'cartao_credito';
  description: string;
  externalRef: string;
  payerEmail: string;
  cardToken?: string; // Para cartão de crédito
  installments?: number;
}

export async function createPaymentCharge(input: ChargeInput) {
  const payment = new Payment(client);

  const body: any = {
    transaction_amount: input.amount,
    description: input.description,
    external_reference: input.externalRef,
    payer: { email: input.payerEmail },
  };

  if (input.method === 'pix') {
    body.payment_method_id = 'pix';
  } else {
    body.payment_method_id = 'credit_card';
    body.token = input.cardToken;
    body.installments = input.installments ?? 1;
  }

  const result = await payment.create({ body });
  return result;
}

// Webhook Handler
// src/app/api/webhooks/mercadopago/route.ts
import { NextRequest, NextResponse } from 'next/server';
import { createServerClient } from '@lib/supabase/server';

export async function POST(request: NextRequest) {
  const supabase = createServerClient();
  const body = await request.json();

  // Validar assinatura do webhook
  const signature = request.headers.get('x-signature');
  // TODO: Validar HMAC com MERCADO_PAGO_WEBHOOK_SECRET

  if (body.type === 'payment' && body.data?.id) {
    const paymentId = body.data.id.toString();
    const MercadoPagoConfig = (await import('mercadopago')).default;
    const { Payment } = await import('mercadopago');
    const mpClient = new MercadoPagoConfig({ accessToken: process.env.MERCADO_PAGO_ACCESS_TOKEN! });
    const payment = new Payment(mpClient);
    const paymentData = await payment.get({ id: paymentId });
  }
}
```

```
if (paymentData.status === 'approved') {
  const bookingId = paymentData.external_reference;
  await supabase.from('bookings').update({
    payment_status: 'pago',
    amount_paid: paymentData.transaction_amount,
    status: 'confirmada',
  }).eq('id', bookingId);
} else if (paymentData.status === 'cancelled' || paymentData.status ===
'rejected') {
  await supabase.from('bookings')
    .update({ payment_status: 'falhou' })
    .eq('gateway_payment_id', paymentId);
}

return NextResponse.json({ received: true });
}
```

```
// src/lib/payments/asaas.ts
const ASAAS_API_URL = process.env.ASAAS_API_URL ?? 'https://api.asaas.com/v3';
const ASAAS_API_KEY = process.env.ASAAS_API_KEY!;

interface AsaasChargeInput {
  customerId: string; // ID do cliente no Asaas (criar se não existir)
  amount: number;
  dueDate: string; // 'YYYY-MM-DD'
  description: string;
  billingType: 'PIX' | 'CREDIT_CARD';
  externalRef: string;
}

async function asaasFetch(path: string, options: RequestInit = {}) {
  const res = await fetch(`${ASAAS_API_URL}${path}`, {
    ...options,
    headers: {
      'Content-Type': 'application/json',
      'access_token': ASAAS_API_KEY,
      ...(options.headers ?? {}) as Record<string, string>,
    },
  });
  if (!res.ok) throw new Error(`Asaas error ${res.status}: ${await res.text()}`);
  return res.json();
}

export async function createAsaasCustomer(name: string, cpf: string, email: string) {
  return asaasFetch('/customers', {
    method: 'POST',
    body: JSON.stringify({ name, cpfCnpj: cpf, email }),
  });
}

export async function createAsaasCharge(input: AsaasChargeInput) {
  return asaasFetch('/payments', {
    method: 'POST',
    body: JSON.stringify({
      customer: input.customerId,
      billingType: input.billingType,
      value: input.amount,
      dueDate: input.dueDate,
      description: input.description,
      externalReference: input.externalRef,
    }),
  });
}

// Buscar QR Code PIX de uma cobrança Asaas
export async function getAsaasPixQrCode(paymentId: string) {
  return asaasFetch(`/payments/${paymentId}/pixQrCode`);
}

// Webhook Handler Asaas
// src/app/api/webhooks/asaas/route.ts
import { NextRequest, NextResponse } from 'next/server';
import { createServerClient } from '@lib/supabase/server';

export async function POST(request: NextRequest) {
  const supabase = createServerClient();
  // Validar token do webhook
  const token = request.headers.get('asaas-webhook-token');
  if (token !== process.env.ASAAS_WEBHOOK_TOKEN) {

```

```
    return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });
  }

  const body = await request.json();
  const { event, payment } = body;

  if (event === 'PAYMENT_RECEIVED' || event === 'PAYMENT_CONFIRMED') {
    await supabase.from('bookings').update({
      payment_status: 'pago',
      amount_paid: payment.value,
      status: 'confirmada',
    }).eq('gateway_payment_id', payment.id);
  }

  return NextResponse.json({ received: true });
}
```

16. Portal Pós-Reserva do Cliente

```
// src/app/minha-reserva/[bookingRef]/page.tsx
import { createServerClient } from '@lib/supabase/server';
import { notFound, redirect } from 'next/navigation';
import Image from 'next/image';
import { CheckCircle, Clock, MapPin, Camera, Download, Star } from 'lucide-react';

export default async function PostBookingPortal({ params }: { params: { bookingRef: string } }) {
  const supabase = createServerClient();
  const { data: { user } } = await supabase.auth.getUser();
  if (!user) redirect('/auth/login?next=/minha-reserva/' + params.bookingRef);

  const { data: booking } = await supabase
    .from('bookings')
    .select(`
      *,
      tour:tours(*),
      schedule:tour_schedules(*),
      agency:agencies(name, logo_url, hq_google_maps_url, hq_waze_url),
      waivers(status, signed_at)
    `)
    .eq('booking_ref', params.bookingRef)
    .eq('client_id', user.id)
    .single();

  if (!booking) notFound();

  const scheduleDate = new Date(booking.schedule.schedule_date + 'T00:00:00');

  return (
    <main className="max-w-lg mx-auto px-4 py-8 space-y-6">
      {/* Header */}
      <div className="text-center">
        <div
          className="inline-flex items-center justify-center w-16 h-16 bg-green-100 rounded-full mb-4">
          <CheckCircle className="w-9 h-9 text-green-600" />
        </div>
        <h1 className="text-2xl font-extrabold text-stone-800">Reserva {booking.booking_ref}</h1>
        <p className="text-stone-500 mt-1 capitalize">
          Status: <span className={`font-semibold ${
            booking.status === 'confirmada' ? 'text-green-600' :
            booking.status === 'pendente' ? 'text-amber-600' : 'text-stone-600'
          }`}>{booking.status}</span>
        </p>
      </div>

      {/* Detalhes do passeio */}
      <div className="bg-white rounded-2xl shadow-sm border border-stone-100 p-5 space-y-3">
        {booking.agency.logo_url && (
          <Image src={booking.agency.logo_url} alt={booking.agency.name} width={80} height={40}
            className="object-contain h-10 w-auto" />
        )}
        <h2 className="font-bold text-stone-800 text-lg">{booking.tour.title}</h2>
        <div className="text-stone-600 text-sm space-y-1.5">
          <p className="flex items-center gap-2">
            <Clock className="w-4 h-4 text-amber-500" />
            {scheduleDate.toLocaleDateString('pt-BR', { weekday: 'long', day: '2-digit', month: 'long' })}
          </p>
        </div>
      </div>
    </main>
  );
}
```

```

        &nbsp;às {booking.schedule.start_time.slice(0, 5)}
    </p>
    <p className="flex items-center gap-2">
        <MapPin className="w-4 h-4 text-amber-500" />
        {booking.agency.name}
    </p>
</div>

    {/ * Navegação */}
    <div className="flex gap-2 pt-2">
        {booking.agency.hq_google_maps_url && (
            <a href={booking.agency.hq_google_maps_url} target="_blank" rel="noopener
noreferrer"
                className="flex-1 text-center bg-blue-50 text-blue-700 font-medium text-
sm py-2 rounded-xl hover:bg-blue-100 transition">
                    Google Maps
                </a>
            )}
        {booking.agency.hq_waze_url && (
            <a href={booking.agency.hq_waze_url} target="_blank" rel="noopener nore-
ferrer"
                className="flex-1 text-center bg-sky-50 text-sky-700 font-medium text-sm py-2 rounded-
xl hover:bg-sky-100 transition">
                    Waze
                </a>
            )}
    </div>
</div>

    {/ * Termo de Responsabilidade */}
    <div className={`rounded-2xl border p-4 ${
        booking.waivers?.[0]?.status === 'assinado'
        ? 'bg-green-50 border-green-200'
        : 'bg-amber-50 border-amber-200'
    }`>
        <p className="font-medium text-sm">
            {booking.waivers?.[0]?.status === 'assinado'
                ? '✅ Termo assinado em ' + new Date(booking.waivers[0].signed_at!).toLoc-
aleDateString('pt-BR')
                : '⚠️ Termo de Responsabilidade pendente de assinatura'}
        </p>
    </div>

    {/ * Galeria de Fotos Pós-Passeio */}
    {booking.photo_gallery_url && (
        <div className="bg-white rounded-2xl shadow-sm border border-stone-100 p-5
space-y-3">
            <div className="flex items-center gap-2 font-bold text-stone-800">
                <Camera className="w-5 h-5 text-amber-500" />
                Suas Fotos e Vídeos
            </div>
            <p className="text-stone-600 text-sm">0 álbum do seu passeio está
disponível para download.</p>
            <a href={booking.photo_gallery_url} target="_blank" rel="noopener norefer-
rer"
                className="flex items-center justify-center gap-2 bg-amber-500 hover:bg-
amber-600 text-white font-bold py-3 rounded-xl transition">
                <Download className="w-5 h-5" />
                Baixar Fotos e Vídeos
            </a>
        </div>
    )}

```

```

    {/* Deixar Avaliação */}
    {booking.status === 'concluida' && (
      <div className="bg-white rounded-2xl shadow-sm border border-stone-100 p-5
space-y-3">
        <div className="flex items-center gap-2 font-bold text-stone-800">
          <Star className="w-5 h-5 text-amber-500" />
          Avalie seu Passeio
        </div>
        <p className="text-stone-600 text-sm">Sua opinião ajuda outros aventureiros
a escolherem.</p>
        <a href={` /agencia/${booking.agency.slug} ?? ''}?review=${booking.id}`}
          className="block text-center bg-stone-800 hover:bg-stone-900 text-white
font-bold py-3 rounded-xl transition">
          Deixar Avaliação
        </a>
      </div>
    )}
  </main>
);
}

```

17. Configuração PWA — Suporte Offline do Guia

```

// public/manifest.json
{
  "name": "RotaBase Guia",
  "short_name": "RTB Guia",
  "description": "Painel offline do guia de passeios",
  "start_url": "/dashboard/guia",
  "display": "standalone",
  "background_color": "#1c1917",
  "theme_color": "#f59e0b",
  "orientation": "portrait",
  "icons": [
    { "src": "/icons/icon-192.png", "sizes": "192x192", "type": "image/png" },
    { "src": "/icons/icon-512.png", "sizes": "512x512", "type": "image/png" },
  ],
  "purpose": "any maskable"
}

```

```
// next.config.ts
import type { NextConfig } from 'next';
const withPWA = require('next-pwa')({
  dest: 'public',
  register: true,
  skipWaiting: true,
  disable: process.env.NODE_ENV === 'development',
  runtimeCaching: [
    {
      urlPattern: /^https:\/\/.*\.supabase\.co\/.*\/i,
      handler: 'NetworkFirst',
      options: {
        cacheName: 'supabase-cache',
        expiration: { maxAgeSeconds: 24 * 60 * 60 }, // 24h
      },
    },
  ],
});

const nextConfig: NextConfig = {
  images: {
    remotePatterns: [
      { protocol: 'https', hostname: '*.supabase.co' },
      { protocol: 'https', hostname: '*.supabase.in' },
    ],
  },
};

module.exports = withPWA(nextConfig);
```

```
// src/lib/supabase/client.ts
import { createBrowserClient as createSupabaseBrowserClient } from '@supabase/ssr';

export function createBrowserClient() {
  return createSupabaseBrowserClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!
  );
}
```

```
// src/lib/supabase/server.ts
import { createServerClient as createSupabaseServerClient } from '@supabase/ssr';
import { cookies } from 'next/headers';

export function createServerClient() {
  const cookieStore = cookies();
  return createSupabaseServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
        getAll: () => cookieStore.getAll(),
        setAll: (cookiesToSet) => {
          try {
            cookiesToSet.forEach(({ name, value, options }) =>
              cookieStore.set(name, value, options)
            );
          } catch {}
        },
      },
    },
  );
}
```

```
/* src/app/globals.css – Classes utilitárias personalizadas */
@tailwind base;
@tailwind components;
@tailwind utilities;

@layer components {
  .input-base {
    @apply w-full border border-stone-200 rounded-xl px-4 py-2.5 text-sm
      focus:outline-none focus:border-amber-400 focus:ring-2 focus:ring-amber-100
      transition bg-white;
  }
}
```

Dependências Principais

```
// package.json (dependências relevantes)
{
  "dependencies": {
    "next": "^14.2.0",
    "react": "^18.3.0",
    "react-dom": "^18.3.0",
    "@supabase/supabase-js": "^2.45.0",
    "@supabase/ssr": "^0.5.0",
    "mercadopago": "^2.2.0",
    "leaflet": "^1.9.4",
    "next-pwa": "^5.6.0",
    "lucide-react": "^0.400.0",
    "tailwindcss": "^3.4.0",
    "typescript": "^5.5.0"
  },
  "devDependencies": {
    "@types/leaflet": "^1.9.12",
    "@types/node": "^20.0.0",
    "@types/react": "^18.3.0"
  }
}
```

Resumo da Arquitetura

Camada	Tecnologia	Responsabilidade
Frontend	Next.js 14 App Router + Tailwind CSS	UI pública, painel empresa, guia PWA
Backend	Next.js Server Actions + Route Handlers	Lógica de negócio, webhooks, auth
Banco de Dados	Supabase (PostgreSQL)	Schema, RLS, Triggers, Views
Autenticação	Supabase Auth	JWT, RBAC por role
Pagamentos	Mercado Pago / Asaas	PIX + Cartão de Crédito
WhatsApp	Evolution API / Z-API	Notificações automáticas
Mapas	Leaflet + OpenStreetMap	Sem custo, sem API Key
Offline	IndexedDB + next-pwa	Painel do guia funciona sem internet
Storage	Supabase Storage	Fotos da frota, PDFs de termos

